UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME TÉCNICO DE INVESTIGACIÓN FORMATIVA

"RUTAS MÍNIMAS CON DISTINTAS COLAS DE PRIORIDAD EN REDES VIALES URBANAS DE LA CIUDAD DEL CUSCO"

Asignatura: ALGORITMOS AVANZADOS (Semestre 2026-2)

Docente Responsable: Ing. Héctor Eduardo Ugarte Rojas

Identificador: GRUPO 5

Integrantes:

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

Cusco – Perú – 2026

Resumen

Se comparó Dijkstra con tres colas de prioridad implementadas en Python: heap binario indexado, heap 4-ario y Fibonacci Heap. El banco sintético conserva 1,350 mediciones de tiempo en diez escalas dispersas y densas. El caso aplicado utiliza un *snapshot* vial de OpenStreetMap de Cusco con 891 vértices y 2,139 aristas dirigidas; sus pesos representan tiempos nominales, sin pendientes ni congestión medidas por tramo. Antes de medir cada instancia sintética se compararon las distancias entre colas y frente a NetworkX, y se verificó una solución con el certificado de optimalidad. La suite prueba Bellman-Ford por separado.

El 4-ario registró la menor mediana en siete de las diez escalas sintéticas; el binario, en las otras tres y en la consulta OSM (15.65 ms frente a 19.87 ms del 4-ario). Los IQR de ambos se solapan en todos esos escenarios. Fibonacci tuvo la mayor mediana en las diez escalas, pese a su mejor cota amortizada para `Decrease-Key`. En la consulta OSM, A^* expandió 7.2 % menos vértices que Dijkstra y conservó el costo de viaje. Estas mediciones describen las instancias y el entorno estudiados; no se midieron fallos de caché ni se obtuvo un ganador aplicable a cualquier red.

Palabras clave: Algoritmo de Dijkstra, colas de prioridad, heap binario, heap 4-ario, Fibonacci heap, rutas mínimas, OpenStreetMap, Cusco, A^*, análisis de algoritmos.

Abstract

We compared three Python priority queues for Dijkstra: an indexed binary heap, a 4-ary heap, and a Fibonacci heap. The synthetic benchmark stores 1,350 timing samples across ten sparse and dense graph scales. The applied case uses an OpenStreetMap snapshot of Cusco with 891 vertices and 2,139 directed edges; its nominal travel times do not include measured slopes or traffic per road segment. Before timing each synthetic graph, the three solutions were compared with each other and with NetworkX, and one solution was checked with an optimality certificate. Bellman-Ford is tested separately in the automated suite.

The 4-ary heap had the lowest median in seven of the ten synthetic scales. The binary heap led in the other three and in the OSM query (15.65 ms versus 19.87 ms for the 4-ary heap). Their interquartile ranges overlap in all these cases. Fibonacci had the highest median at every synthetic scale despite its better amortized bound for `Decrease-Key`. For the OSM query, A^* expanded 7.2 % fewer vertices than Dijkstra at the same travel cost. These observations apply to the tested graphs and machine; cache misses were not measured.

Keywords: Dijkstra's algorithm, priority queues, binary heap, 4-ary heap, Fibonacci heap, shortest paths, OpenStreetMap, Cusco, A^*, experimental analysis.

Índice

Resumen I

Abstract II

1. Introducción 1
1.1. Contexto y Motivación . . . 1
1.2. Planteamiento del Problema . . . 1
1.3. Objetivos del Proyecto . . . 1
1.3.1. Objetivo General . . . 1
1.3.2. Objetivos Específicos . . . 1
1.4. Alcance y Delimitación . . . 2
1.5. Control de Versiones y Repositorio Oficial en GitHub . . . 2
1.6. Organización del Documento . . . 3

2. Fundamentos y Aprendizaje Autónomo 4
2.1. Conceptos y Algoritmos Requeridos . . . 4
2.1.1. Grafos Viales Urbanos y Propiedad de Red Cuasi-Plana . . . 4
2.1.2. Algoritmo de Dijkstra e Invariante de Subestructura Óptima . . . 4
2.1.3. Estructuras de Colas de Prioridad con Decrease-Key . . . 4
2.1.4. Extensión Algorítmica: Búsqueda Heurística A^* y Poda Espacial . . . 5
2.1.5. Metodología de Pruebas Unitarias: Definición Formal del Patrón Arrange-Act-Assert (AAA) 6
2.1.6. Teoría de Oráculos de Prueba y Verificación Formal de Corrección . . . 6
2.2. Fuentes Estudiadas y Criterios para Seleccionarlas . . . 6
2.2.1. Criterios de Selección de Fuentes Académicas y Técnicas (C1 a C5) . . . 6
2.2.2. Matriz Razonada de Fuentes Bibliográficas Clave . . . 7
2.3. Conocimientos Nuevos que el Grupo Tuvo que Adquirir . . . 8
2.3.1. Metodología de Pruebas Unitarias AAA y Oráculos de Verificación . . . 8
2.3.2. Manipulación Dinámica de Estructuras Arbóreas con Múltiples Punteros . . . 8
2.3.3. Cálculo de Costos Amortizados y Mecanismo de Cortes en Cascada . . . 8
2.3.4. Cotas amortizadas y tiempos observados . . . 9
2.3.5. Modelado de Redes Viales Reales y Normalización con OSMnx . . . 9
2.4. Estrategia y Evidencias de Aprendizaje Autónomo . . . 9
2.4.1. Hito 1: Investigación y Prototipado del Min-Heap Binario Indexado . . . 9
2.4.2. Hito 2: Extensión 4-aria . . . 9
2.4.3. Hito 3: Fibonacci Heap y consolidación . . . 9
2.4.4. Hito 4: Verificación Integral mediante Suite AAA y Certificado de Bellman . . . 9
2.4.5. Evidencias Verificables de Logro del Atributo AG_C06 . . . 10

3. Evaluación y Selección del Stack Tecnológico 11
3.1. Alternativas Consideradas . . . 11
3.2. Criterios de Comparación . . . 11
3.3. Tecnología Seleccionada y Justificación . . . 11
3.4. Procedimiento de Aprendizaje e Integración . . . 12
3.5. Dificultades de Adopción y Soluciones Aplicadas . . . 13

4. Formulación del Problema 14
4.1. Definición Formal del Grafo Vial Urbano . . . 14

4.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema . . . 14
4.2.1. Variables de Decisión Binarias . . . 14
4.2.2. Salidas del Sistema . . . 14
4.3. Restricciones de Conservación de Flujo y No Negatividad . . . 15
4.4. Función Objetivo y Modelo de Costo con Impedancia Andina . . . 15
4.4.1. Formulación de la Impedancia Vial Andina . . . 15
4.4.2. Variables Meteorológicas . . . 16
4.4.3. Modelo de Congestión Horaria Hipotético . . . 16
4.4.4. Supuestos del Modelo . . . 17
4.5. Ejemplo Ilustrativo Resuelto Analíticamente (Red Nuclear de 6 Nodos) . . . 17
4.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$) . . . 18
4.7. Instancia de verificación canónica de CLRS . . . 19
4.8. Red Vial Urbana Real de Cusco (OpenStreetMap) . . . 20

5. Diseño Algorítmico **21**
5.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta . . . 21
5.1.1. Correspondencia entre Pseudocódigo e Implementación . . . 21
5.2. Invariantes de Bucle y Demostración Formal de Corrección . . . 22
5.2.1. Invariante de Bucle Fundamental . . . 22
5.2.2. Demostración por Inducción Matemática de Optimalidad . . . 22
5.3. Estructuras de Datos de Cola de Prioridad Evaluadas . . . 23
5.3.1. Min-Heap Binario Indexado (`IndexedMinHeap`) . . . 23
5.3.2. 4-ary Heap Contiguo (`DAryHeap`, $d = 4$) . . . 23
5.3.3. Fibonacci Heap (`FibonacciHeap`) . . . 23
5.4. Análisis Comparativo de Complejidades Teóricas . . . 24
5.5. Guía Práctica de Selección según el Escenario . . . 24

6. Diseño e Implementación del Sistema **26**
6.1. Arquitectura Modular del Sistema (4 Capas) . . . 26
6.2. Descripción Detallada de Módulos y Componentes . . . 26
6.3. Contrato Unificado e Integración de la Aplicación Móvil (`mobile/`) . . . 27
6.3.1. Arquitectura y Diseño de la Aplicación Móvil Flutter . . . 28
6.3.2. Módulos de la Aplicación Móvil . . . 28
6.4. Formatos y Estructura de Datos (Serialización JSON) . . . 31
6.5. Contratos de Interfaz e Inyección de Colas de Prioridad . . . 33
6.6. Manejo Defensivo de Entradas y Validación . . . 33
6.7. Servicios Complementarios: Meteorología y Escenarios de Tráfico . . . 33
6.8. Decisiones de Implementación Relevantes y Justificación Técnica . . . 34

7. Estrategia de Pruebas **35**
7.1. Framework de Pruebas y Patrón Arrange-Act-Assert (AAA) . . . 35
7.2. Tipología y Cobertura de la Suite Automatizada (349 Pruebas Python) . . . 35
7.3. Pruebas Multiplataforma (Web Vitest y Móvil Flutter) . . . 36
7.4. Metodología de Verificación y Oráculos Independientes . . . 37
7.5. Certificado de Optimalidad de Bellman . . . 37
7.6. Casos Límite, Condiciones Adversas y Pruebas de Regresión . . . 38
7.7. Pruebas de Integración y Funcionamiento Desconectado . . . 38
7.8. Criterios de Aceptación y Calidad del Software . . . 38

8. Metodología Experimental **39**
8.1. Preguntas de Investigación e Hipótesis de Evaluación . . . 39
8.1.1. Hipótesis de Trabajo de la Evaluación (H1 a H4) . . . 39
8.2. Diseño Factorial e Instancias Experimentales . . . 39
8.2.1. Generación Determinista de Instancias Sintéticas . . . 40
8.3. Variables Experimentales y Métricas de Desempeño . . . 40
8.4. Entorno Experimental Controlado e Instrumentación de Precisión . . . 40
8.4.1. Protocolo de Aislamiento de Ruido y Precisión Temporal . . . 41
8.5. Caso Aplicado sobre la Red Vial Urbana Real de Cusco . . . 41
8.6. Procedimiento Experimental Reproducible . . . 42
8.7. Piloto de Verificación del Protocolo . . . 42
8.8. Amenazas a la Validez . . . 43

9. Resultados **44**
9.1. Verificación de Corrección y Equivalencia de Distancias . . . 44
9.2. Benchmark Canónico en Instancias Sintéticas (Perfiles Disperso y Denso) . . . 44
9.2.1. Análisis del Perfil Disperso ($|E| = 4|V|$) . . . 45
9.2.2. Análisis del Perfil Denso ($|E| = 0.08\,|V|(|V|-1)$) . . . 45
9.2.3. Desempeño Comparativo del Fibonacci Heap . . . 45
9.3. Análisis Cuantitativo de Operaciones Elementales y Comparaciones . . . 46
9.4. Caso de Estudio Aplicado: Red Vial Real de Cusco (OpenStreetMap) . . . 47
9.5. Extensión Algorítmica con Búsqueda Heurística A^* . . . 50
9.6. Ampliación del Caso de Estudio: Escala Urbana (1.5 km vs. 3.0 km) e Integración de Condiciones 50

10. Discusión Crítica **53**
10.1. Contraste entre Teoría Asintótica y Desempeño Empírico . . . 53
10.2. Montículo 4-ario: cotas y mediciones . . . 53
10.3. Desempeño del Montículo Binario en la Red Real de Cusco . . . 53
10.4. Influencia de la Densidad del Grafo y Operaciones Decrease-Key . . . 53
10.5. Evaluación de la Poda Espacial con Búsqueda Heurística A^* . . . 53
10.6. Análisis de Sensibilidad de Factores Ambientales e Invariancia de Ruta . . . 54
10.7. Guía Práctica y Marco de Decisión para Selección de Colas . . . 54
10.8. Limitaciones del Estudio y Amenazas a la Validez . . . 54

11. Conclusiones y Recomendaciones **55**

12. Referencias **57**

13. Anexos **58**

Índice de figuras

1. Instancia didáctica del Centro Histórico de Cusco ($|V| = 6$, $|E| = 9$). Longitudes esquemáticas en metros. . . . 17
2. Agrupación funcional de módulos; las flechas superiores indican llamadas, no un flujo exclusivo de datos . . . 26
3. Módulo de cálculo de Rutas en la aplicación móvil Flutter (`mobile/`). A la izquierda, visualización de la ruta mínima calculada sobre el plano vial de Cusco con controles flotantes de interacción, zoom y bloqueo. A la derecha, desglose cuantitativo con distinción entre tiempo del solver en servidor y latencia de red, junto al contraste de las cuatro variantes algorítmicas. . . . 30
4. Módulos de Análisis y Ajustes en la aplicación móvil Flutter. A la izquierda, inspección visual de las medianas e IQR del banco canónico de 1 350 corridas. A la derecha, estado de conectividad en vivo con el backend FastAPI, guía de operación sin internet y panel «Acerca de» con versión instalada y autoría académica del Grupo 5 (UNSAAC). . . . 31
5. Piloto del 04/11/2026. Cada punto representa una medición; el segmento, la mediana de tres repeticiones. Datos: `pilot_benchmark_results.json`. La comparación por escenario se presenta en la Sección 9. . . . 42
6. Tiempo mediano de ejecución de Dijkstra con las tres colas de prioridad evaluadas. Izquierda: perfil disperso ($|E| = 4|V|$). Derecha: perfil denso ($|E| = 0.08\,|V|(|V| - 1)$). Las barras de error representan el Intervalo Intercuartílico (IQR). . . . 45
7. Distribución de operaciones elementales de Dijkstra en el perfil disperso sintético. Se discriminan inserciones, extracciones, decrementos de clave efectivos e intentos totales de relajación. . . . 47
8. Ruta de menor tiempo nominal sobre el recorte OpenStreetMap de Cusco. La línea representa el camino calculado y los marcadores señalan origen y destino. . . . 48
9. Vista de referencia del Centro Histórico del Cusco en OpenStreetMap. La imagen se utiliza para identificar visualmente el área de estudio y no representa la ruta calculada por el programa. . . . 49
10. Integración de modos de escala e interfaces de condiciones ambientales en la aplicación móvil Flutter. A la izquierda, alternancia rápida entre el recorte de 1.5 km y la red ampliada de 3.0 km con controles de búsqueda y visualización. A la derecha, configuración modular del escenario climático (Open-Meteo) y congestión horaria programada. . . . 52

Índice de tablas

1. Selección razonada de fuentes bibliográficas clave y criterios de pertinencia . . . 8
2. Alternativas por capa tecnológica . . . 11
3. Matriz multicriterio del lenguaje principal (escala 1–5) . . . 11
4. Velocidades supuestas para la instancia didáctica; no usadas en la consulta OSM . . . 16
5. Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco. . . . 18
6. Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo. . . . 19
7. Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor . . . 19
8. Correspondencia entre pseudocódigo y la implementación en Python . . . 22
9. Cotas temporales y espaciales de las colas de prioridad evaluadas . . . 24
10. Guía de selección práctica de algoritmos y estructuras en redes viales . . . 25
11. Archivos de datos principales y su rol funcional en el sistema . . . 32
12. Cobertura de las pruebas automatizadas por categoría funcional . . . 35
13. Distribución modular de la suite Python de 349 pruebas . . . 36
14. Pruebas automatizadas de los clientes Web (44) y Móvil Flutter (40) . . . 37

15. Especificaciones del entorno experimental de medición 41
16. Dijkstra con tres colas, perfil disperso: mediana e IQR en μs. 44
17. Dijkstra con tres colas, perfil denso: mediana e IQR en μs. 44
18. Síntesis por perfil y consulta: conteo de menores medianas observadas, sin declarar diferencias concluyentes donde se solapan los IQR. 46
19. Caso OSM: tiempo de Dijkstra por cola, mediana e IQR. 49
20. Sensibilidad meteorológica uniforme en la consulta OSM. 49
21. Comparación entre el recorte central (1.5 km) y la red urbana ampliada (3.0 km) de Cusco 50
22. Hitos fechados comprobables en la historia de Git (`main`) 61
23. Anexo D: matriz de trazabilidad entre objetivos, componentes y evidencia 62
24. Distribución verificable de los 122 commits en `main` por autoría Git y rama asociada 62
25. Matriz de componentes por integrante y evidencia en el repositorio 63

1. Introducción

1.1. Contexto y Motivación

Para estudiar rutas mínimas en una red vial no basta con conocer la distancia entre dos puntos: también importan el sentido de las calles y el costo asignado a cada tramo. En este trabajo comparamos el tiempo de cómputo de tres colas de prioridad dentro de Dijkstra. La red de Cusco sirve como caso de estudio, no como prueba de tiempos de respuesta de servicios de emergencia.

Cusco aporta un ejemplo concreto de calles dirigidas. Se trabajó con una instancia didáctica de seis nodos, cuyas longitudes y pendientes son supuestos, y con un recorte OSM para conducción, que incluye sentidos y longitudes pero no pendientes ni tráfico medidos por arista. Esta separación evita atribuir al caso OSM condiciones que el archivo no registra.

1.2. Planteamiento del Problema

En este trabajo usamos el algoritmo de Dijkstra (Dijkstra, 1959) para resolver el problema de caminos mínimos desde un origen único (*Single-Source Shortest Path*, SSSP) en grafos dirigidos con pesos no negativos ($w(e) \geq 0$). La complejidad asintótica depende de la estructura de datos que usamos para administrar los vértices tentativos:

$$T(|V|,|E|) = O(|V| \cdot T_{\text{Extract-Min}} + |E| \cdot T_{\text{Decrease-Key}} + |V| \cdot T_{\text{Insert}}) \tag{1}$$

La cota amortizada de $O(|E| + |V| \log |V|)$ para Fibonacci Heap se basa en Fredman y Tarjan (1987). La contrastamos con tiempos de ejecución y conteos de operaciones en Python. No instrumentamos asignaciones ni accesos a caché; las mediciones y el equipo se describen en la Sección 8.

El problema de investigación es cómo cambia el tiempo de Dijkstra al sustituir la cola, manteniendo la misma instancia y el mismo algoritmo. Se comparan grafos sintéticos y una consulta vial OSM con tiempo nominal de viaje.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Implementar y comparar Dijkstra con heap binario indexado, heap 4-ario y Fibonacci Heap en grafos sintéticos y en una consulta OSM de Cusco. Se miden tiempos y operaciones y se comprueban las distancias; no se reporta consumo de memoria medido.

1.3.2. Objetivos Específicos

Siguiendo la metodología de ingeniería de software y análisis de algoritmos, los objetivos específicos se ordenan en cuatro fases secuenciales e integradas:

1. **OE1 (Modelar):** Diferenciar el grafo didáctico con pendientes supuestas del *snapshot* OSM con tiempos de viaje nominales y sentidos viales.

2. **OE2 (Implementar):** Construir Dijkstra con tres colas intercambiables y una extensión A^* para consultas punto a punto.

3. **OE3 (Verificar):** Validar analíticamente la correctitud del algoritmo mediante traza manual paso a paso, certificado formal de optimalidad independiente en $O(|V|+|E|)$ y una suite automatizada de pruebas unitarias bajo el patrón Arrange-Act-Assert (AAA), contrastando los resultados contra oráculos canónicos de referencia (NetworkX y Bellman-Ford).

4. **OE4 (Evaluar):** Registrar tiempos de pared y operaciones en redes sintéticas y en una consulta OSM; comparar los resultados con las cotas teóricas, sin atribuir diferencias a caché no medida.

1.4. Alcance y Delimitación

El proyecto se delimita formalmente en las siguientes dimensiones:

- **Delimitación Espacial:** Analizamos dos escalas complementarias: (1) la malla vial nuclear del Centro Histórico del Cusco, empleada para la traza manual y la verificación paso a paso; y (2) la red vial del área central de Cusco, que extrajimos de OpenStreetMap mediante OSMnx (Boeing, 2025), con un radio de 1,500 m, 891 vértices y 2,139 aristas dirigidas. La web permite explorar además un snapshot radial de 3 km, independiente y no usado en el banco canónico.

- **Delimitación Algorítmica:** Se evalúa el algoritmo de Dijkstra en grafos dirigidos estáticos con costos deterministas no negativos ($w(e) \geq 0$). Como extensión de ruteo punto a punto se integra A^* con heurística consistente. Bellman-Ford y NetworkX operan exclusivamente como oráculos independientes de verificación cruzada.

- **Alcance del Sistema:** Implementamos la formulación matemática, la solución analítica manual, el núcleo ejecutable en Python 3.12 con interfaz de línea de comandos (CLI), la API FastAPI v1.5.0 desplegada en Render, la aplicación web React/TypeScript publicada en GitHub Pages, la aplicación móvil en Flutter/Dart (`mobile/`) con base de datos SQLite interna del dispositivo, el cuaderno didáctico en Google Colab y la suite multiplataforma de 433 pruebas automatizadas (349 Python, 44 web y 40 móvil). No reportamos métricas de consumo de memoria porque no las medimos.

1.5. Control de Versiones y Repositorio Oficial en GitHub

Conforme a las directrices de reproducibilidad y trabajo colaborativo del curso, el desarrollo íntegro del proyecto se gestiona bajo el sistema de control de versiones Git y se aloja en el repositorio público oficial:

https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC

La organización de ramas adopta un esquema estructurado: la rama `main` almacena las versiones estables consolidadas, complementada por ramas de desarrollo temáticas por integrante (`ramaFRANKI`, `ramaAldo`, `ramaYeni`, `RamaRomario`). Los datos experimentales se versionan en `python/data/processed/` para garantizar reproducibilidad total sin requerir descargas en vivo de APIs externas.

1.6. Organización del Documento

El presente informe técnico se organiza conforme a los requerimientos temáticos del apartado 9 del sílabo: La Sección 1 presenta la introducción, contexto y objetivos. La Sección 2 detalla los fundamentos teóricos de grafos viales, colas de prioridad con clave disminuible, búsqueda heurística A^*, la formalización del patrón de pruebas unitarias Arrange-Act-Assert (AAA), oráculos de verificación y el proceso de aprendizaje autónomo. La Sección 3 evalúa y justifica el stack tecnológico mediante una matriz multicriterio. La Sección 4 expone la formulación matemática del problema y el modelo de costo cusqueño. La Sección 5 formaliza el diseño algorítmico, pseudocódigos, invariantes y complejidades. La Sección 6 detalla la arquitectura modular del software y los componentes implementados. La Sección 7 especifica la estrategia de pruebas unitarias (AAA), oráculos y certificado de optimalidad. La Sección 8 formula las hipótesis, variables y el protocolo de experimentación reproducible. La Sección 9 documenta los resultados computacionales del benchmark canónico y la red real OSM. La Sección 10 contrasta cotas y mediciones, incluidas las limitaciones para explicar las causas de hardware. La Sección 11 sintetiza las conclusiones en respuesta a los objetivos planteados y propone trabajo futuro. La Sección 12 lista las fuentes académicas bajo norma APA 7ma edición. Finalmente, la Sección 13 reúne instrucciones, cronología de Git y matriz de contribuciones; no contiene bitácoras individuales.

2. Fundamentos y Aprendizaje Autónomo

2.1. Conceptos y Algoritmos Requeridos

El proyecto combina grafos dirigidos, Dijkstra con colas que admiten `Decrease-Key`, la búsqueda A^* y pruebas automatizadas. Esta sección describe los conceptos y su uso en el repositorio, sin atribuir el aprendizaje personal de cada integrante.

2.1.1. Grafos Viales Urbanos y Propiedad de Red Cuasi-Plana

Una red vial se representa como un grafo dirigido ponderado $G = (V, E, W)$: los vértices son puntos de conexión, las aristas son tramos permitidos y $W : E \to \mathbb{R}_{\geq 0}$ asigna un costo no negativo. La relación entre aristas y vértices depende del recorte y de cómo se representan las intersecciones. En el caso OSM se midió:

$$|E|/|V| = 2139/891 \approx 2.40. \tag{2}$$

El grafo sintético disperso tiene cuatro aristas por vértice; el denso se generó con aproximadamente el 8 % de las aristas dirigidas posibles. Estas densidades se comparan con el recorte OSM sin inferir a partir de ellas accesos físicos a memoria.

2.1.2. Algoritmo de Dijkstra e Invariante de Subestructura Óptima

El algoritmo de Dijkstra (Dijkstra, 1959) resuelve el problema de caminos mínimos desde un origen único (*Single-Source Shortest Path*, SSSP) en grafos con pesos no negativos ($w(e) \geq 0$). El algoritmo explora el espacio de estados manteniendo una partición de los vértices en dos conjuntos disjuntos: el conjunto S de vértices cuya distancia mínima desde el origen s ya ha sido permanentemente determinada, y el conjunto $Q = V \setminus S$ de vértices tentativos gestionados dinámicamente en una cola de prioridad. En cada iteración, el algoritmo extrae el vértice $u \in Q$ con la menor distancia tentativa acumulada $d[u]$, lo incorpora a S, y relaja cada arista saliente $(u, v) \in E$:

$$\text{si } d[u] + w(u,v) < d[v] \implies d[v] \leftarrow d[u] + w(u,v), \quad \pi[v] \leftarrow u \tag{3}$$

La actualización del valor $d[v]$ en la cola de prioridad exige ejecutar la operación de disminución de clave `Decrease-Key`$(Q, v, d[v])$.

2.1.3. Estructuras de Colas de Prioridad con Decrease-Key

La cota de complejidad temporal de Dijkstra está gobernada por las tres operaciones elementales de la cola de prioridad:

$$T(|V|, |E|) = O(|V| \cdot T_{\text{Extract-Min}} + |E| \cdot T_{\text{Decrease-Key}} + |V| \cdot T_{\text{Insert}}) \tag{4}$$

En este trabajo se seleccionaron e implementaron puramente tres estructuras con mecánicas internas contrastantes:

- **Min-Heap Binario Indexado (`IndexedMinHeap`):** Estructura arbórea binaria casi completa dispuesta en un arreglo contiguo en memoria, complementada por una tabla hash inversa de posiciones $\text{pos} : V \to \mathbb{N}$ que ubica cualquier identificador de vértice en tiempo $O(1)$. Las operaciones `Insert`, `Extract-Min` y `Decrease-Key` operan en $O(\log |V|)$ mediante flotado (*sift-up*) y hundimiento (*sift-down*).

- **4-ary Heap (`DAryHeap`, $d = 4$):** Sus hijos ocupan índices contiguos $4i+1, \ldots, 4i+4$ en un arreglo. Su profundidad logarítmica es aproximadamente la mitad de la del binario, pero cada extracción puede comparar más hijos. El banco registró tiempos y contadores de operaciones, no accesos a caché.

- **Fibonacci Heap (`FibonacciHeap`):** Colección de árboles con orden de montículo estructurados mediante listas circulares doblemente enlazadas con múltiples punteros por nodo (`parent`, `child`, `left`, `right`, `degree` y `marked`) (Fredman & Tarjan, 1987). Utiliza inserción y corte perezoso (*lazy*), logrando que `Insert` y `Decrease-Key` operen en tiempo constante amortizado $O(1)^*$ mediante el método del potencial de Tarjan (Tarjan, 1983), mientras que `Extract-Min` toma $O(\log |V|)^*$ amortizado mediante consolidación dinámica de árboles de igual grado.

2.1.4. Extensión Algorítmica: Búsqueda Heurística A^* y Poda Espacial

Para una consulta punto a punto, A^* (Hart et al., 1968) ordena los vértices según el costo ya recorrido y una cota del costo restante:

$$f(n) = g(n) + h(n) \tag{5}$$

donde $g(n)$ es el costo real acumulado desde el origen s, y $h(n)$ es una función heurística que estima el costo residual hasta el destino t. Para garantizar matemáticamente la optimalidad de la ruta encontrada, la heurística debe satisfacer:

1. **Admisibilidad:** $0 \leq h(n) \leq h^*(n)$ para todo $n \in V$, donde $h^*(n)$ representa la distancia mínima óptima verdadera hasta t.

2. **Consistencia (Monotonicidad):** $h(u) \leq w(u,v) + h(v)$ para toda arista dirigida $(u,v) \in E$, cumpliendo la desigualdad triangular modificada.

En la red vial de Cusco, con pesos definidos en unidades de tiempo (segundos), la distancia euclidiana física $\|x_u - x_t\|$ se escala mediante el factor admisible:

$$\lambda = \min_{(u,v)\in E} \frac{w(u,v)}{\|x_u - x_v\|}, \quad h(u) = \lambda \cdot \|x_u - x_t\| \tag{6}$$

Con coordenadas y escala compatibles con los pesos, la desigualdad triangular da una heurística consistente. Esto conserva la optimalidad; no garantiza reducir expansiones en todos los grafos.

2.1.5. Metodología de Pruebas Unitarias: Definición Formal del Patrón Arrange-Act-Assert (AAA)

El patrón **Arrange–Act–Assert (AAA)** ayuda a separar preparación, ejecución y comprobación en las pruebas. La suite contiene casos de varios tipos; no se afirma que todos sigan exactamente la misma plantilla.

El patrón ayuda a localizar qué falla en un caso concreto:

Arrange: Se prepara un grafo y su origen; cuando se generan instancias aleatorias, se fija la semilla.

Act: Se invoca una operación de cola o un solver, por ejemplo `dijkstra_solver(nodes, adj, source, queue_type=...)`.

Assert: Se comprueban distancias, invariantes de las colas o propiedades del certificado. La comparación numérica usa tolerancia, no igualdad exacta de valores de coma flotante.

Las entradas fijas permiten repetir los casos, no asegurar tiempos idénticos en equipos distintos. La suite compartida tampoco acredita por sí sola el aprendizaje autónomo individual de AG-C06.01.

2.1.6. Teoría de Oráculos de Prueba y Verificación Formal de Corrección

En la teoría de pruebas de software, un **oráculo de prueba** es un mecanismo independiente y confiable capaz de determinar si la salida de un programa para una entrada dada es correcta o incorrecta. En este proyecto se formalizaron tres oráculos de verificación independientes:

1. **Oráculo analítico (traza manual):** Verificación paso a paso sobre la red nuclear de seis nodos del Centro Histórico del Cusco y sobre la instancia de caminos mínimos presentada por Cormen et al. (2022, figura 24.6, p. 659), comprobando los vectores de distancia y los contenidos de la cola.
2. **Oráculos algorítmicos:** Bellman-Ford se usa en pruebas de la suite; NetworkX (`nx.single_source_dijkstra_path_length`) se contrasta antes del banco sintético. Son comprobaciones en contextos distintos, no dos verificaciones de cada muestra temporal.
3. **Certificado de optimalidad:** Comprueba distancias, aristas y cadenas de predecesores en $O(|V| + |E|)$ cuando no se pasan caminos explícitos:

$$d[s] = 0, \quad d[v] \leq d[u] + w(u,v) \quad \forall (u,v) \in E \tag{7}$$

   También comprueba que cada cadena de predecesores llegue al origen y use aristas ajustadas. Si se proporcionan caminos explícitos, hay que sumar el costo de recorrerlos; las desigualdades de distancias por sí solas no prueban optimalidad.

2.2. Fuentes Estudiadas y Criterios para Seleccionarlas

2.2.1. Criterios de Selección de Fuentes Académicas y Técnicas (C1 a C5)

La selección de la literatura técnica y académica se condujo bajo cinco criterios explícitos de pertinencia, rigor científico y relevancia arquitectónica: Priorizamos fuentes publicadas entre 2021 y

2026 para describir herramientas, datos y prácticas actuales. Conservamos algunas obras anteriores a ese periodo únicamente cuando son las fuentes originales de Dijkstra, Fibonacci Heap o A^*; las citamos como antecedentes fundacionales, no como evidencia contemporánea de rendimiento.

1. **C1: Arbitraje por Pares y Nivel Académico:** Se priorizaron publicaciones seminales indexadas en las principales sociedades científicas (ACM, IEEE, SIAM, Springer), excluyendo blogs informales o tutoriales no validados.

2. **C2: Relevancia Teórica y Demostración Formal:** Inclusión obligatoria de las obras clásicas que demostraron la correctitud y las cotas asintóticas de caminos mínimos y análisis amortizado (Cormen et al., 2022; Dijkstra, 1959; Fredman & Tarjan, 1987; Tarjan, 1983).

3. **C3: Arquitectura de computadores:** Se revisó la diferencia entre cotas y tiempos medidos. El banco no midió localidad física de memoria ni fallos de caché.

4. **C4: Estándares Geoespaciales y Datos Topográficos Abiertos:** Seleccionamos una metodología reciente para ingerir y modelar mallas viales reproducibles desde OpenStreetMap (Boeing, 2025; OpenStreetMap contributors, 2026).

5. **C5: Metodología de Calidad de Software y Reproducibilidad:** Aplicamos el patrón Arrange-Act-Assert y pruebas automatizadas como decisiones metodológicas del equipo; las evidencias de calidad se encuentran en nuestra suite y en el protocolo reproducible.

2.2.2. Matriz Razonada de Fuentes Bibliográficas Clave

La Tabla 1 sistematiza la bibliografía fundamental estudiada y su mapeo contra estos criterios de selección.

Tabla 1
Selección razonada de fuentes bibliográficas clave y criterios de pertinencia

Fuente Académica	Tipología y Dominio	Criterios	Impacto y Aporte al Proyecto
Dijkstra (1959)	Artículo Seminal (Numerische Mathematik)	C1, C2	Fundamento teórico del algoritmo SSSP con pesos no negativos y principio de relajación voraz.
Fredman & Tarjan (1987)	Artículo Teórico (Journal of the ACM)	C1, C2	Especificación formal del Fibonacci Heap con tiempos amortizados $O(1)^*$ para `Decrease-Key`.
Tarjan (1983)	Monografía Clásica (SIAM)	C1, C2	Análisis riguroso del método del potencial para el cálculo de costos amortizados en estructuras de datos.
Hart et al. (1968)	Artículo Seminal (IEEE SSC)	C1, C2	Formulación de A^* y condiciones matemáticas de admisibilidad y consistencia para poda de grafos.
Boeing (2025)	Sistemas Geoespaciales (Geographical Analysis)	C1, C4	Metodología reciente de ingestión, validación y análisis de grafos urbanos desde OpenStreetMap.
Cormen et al. (2022)	Libro de Texto Avanzado (MIT Press)	C1, C2	Demostración formal de invariantes de bucle, cotas asintóticas y pseudocódigo algorítmico estructurado.

2.3. Conocimientos Nuevos que el Grupo Tuvo que Adquirir

En el código y las pruebas se distinguen cinco temas de estudio. La documentación disponible no permite fechar cuándo los identificó cada integrante ni sustituye su registro personal de aprendizaje:

2.3.1. Metodología de Pruebas Unitarias AAA y Oráculos de Verificación

La suite en Python reúne 349 casos unitarios y de integración con `pytest` (complementados por 44 pruebas web en Vitest y 40 pruebas móviles en Flutter). Sus aserciones comprueban distancias frente a Bellman-Ford y NetworkX, así como propiedades del certificado de optimalidad. No todas las pruebas utilizan ambos oráculos.

2.3.2. Manipulación Dinámica de Estructuras Arbóreas con Múltiples Punteros

El Fibonacci Heap utiliza enlaces circulares y referencias `parent`, `child`, `left`, `right`, `degree` y `marked`. Su contrato se comprueba en las pruebas de estructuras; no se midieron fugas de memoria.

2.3.3. Cálculo de Costos Amortizados y Mecanismo de Cortes en Cascada

El análisis amortizado descrito por Tarjan (1983) ayuda a explicar el marcado y los cortes en cascada de `Decrease-Key`. La implementación y las pruebas ejercitan estos casos, sin que ello documente el proceso individual de aprendizaje.

2.3.4. Cotas amortizadas y tiempos observados

En las instancias medidas, el Fibonacci Heap tuvo mayor mediana que las otras colas pese a su cota amortizada para `Decrease-Key`. La menor profundidad teórica del 4-ario es una propiedad de la estructura, no una medición de caché.

2.3.5. Modelado de Redes Viales Reales y Normalización con OSMnx

El cargador OSMnx (Boeing, 2025) guarda nodos con coordenadas WGS84 y tramos dirigidos, incluidas aristas paralelas en el *snapshot*. Al construir la instancia del algoritmo, selecciona el menor peso entre tramos que comparten origen y destino.

2.4. Estrategia y Evidencias de Aprendizaje Autónomo

El historial Git permite ordenar cuatro componentes del desarrollo. Describe cambios de código y documentos, no reuniones ni una planificación original:

2.4.1. Hito 1: Investigación y Prototipado del Min-Heap Binario Indexado

El commit `24e526f` (02/10/2026) incluye Dijkstra y el montículo binario indexado; las referencias de Cormen et al. (2022) y Fredman y Tarjan (1987) sustentan el análisis. No se dispone de una bitácora que feche las lecturas personales.

2.4.2. Hito 2: Extensión 4-aria

El commit `442ea56` (04/10/2026) contiene el heap 4-ario. El banco posterior registra sus operaciones y tiempos frente al binario; no registra fallos de caché.

2.4.3. Hito 3: Fibonacci Heap y consolidación

El commit `221bf21` (04/10/2026) incluye el montículo Fibonacci. El código actual amplía el arreglo de consolidación cuando lo necesita y reserva grados según una cota basada en la razón áurea. Sin un registro fechado del fallo original, no se atribuye a una sesión concreta ni a una resolución individual:

$$D(n) = \lfloor \log_{\phi}(n) \rfloor + 2, \quad \text{donde } \phi = \frac{1+\sqrt{5}}{2} \approx 1.618034 \tag{8}$$

2.4.4. Hito 4: Verificación Integral mediante Suite AAA y Certificado de Bellman

La suite actual en Python contiene 349 casos que cubren contratos de colas, validación de entradas, API HTTP y comprobación de distancias, entre otros módulos. El certificado opera en $O(|V|+|E|)$ para distancias y predecesores, más el costo de recorrer los caminos explícitos cuando se proporcionan. Los resultados sintéticos y OSM se guardaron en el commit `80e2593` del 28/10/2026.

2.4.5. Evidencias Verificables de Logro del Atributo AG_C06

El repositorio permite verificar componentes técnicos, pero no todas las actividades individuales exigidas por el atributo:

- **AG-C06.01 (40 %):** El Anexo C recoge autoría y fechas de commits. Faltan las bitácoras individuales de fuentes, actividades y aprendizajes, así como actas de seguimiento.
- **AG-C06.02 (40 %):** El capítulo de stack describe las herramientas usadas. La instalación desde cero y la reflexión personal de cada estudiante siguen sin acreditarse.
- **AG-C06.03 (20 %):** Los JSON, tablas y discusión permiten contrastar cotas y tiempos observados; los efectos físicos de memoria no fueron medidos.

3. Evaluación y Selección del Stack Tecnológico

3.1. Alternativas Consideradas

La Tabla 2 resume las alternativas evaluadas en seis capas. Se distingue entre la tecnología adoptada para el prototipo y las opciones consideradas para una eventual ampliación. No se valoró la novedad por sí misma, sino la capacidad de aprender, integrar, probar y justificar cada herramienta.

Tabla 2
Alternativas por capa tecnológica

Capa	Alternativas	Decisión	Criterio principal
Lenguaje principal	C++20, Rust 1.75, Python 3.12	**Python 3.12**	Expresividad algorítmica, tipado, instrumentación y rapidez de experimentación.
Red vial y grafos	OSMnx/NetworkX, Overpass API, PostGIS	**OSMnx + NetworkX**	Descarga y conversión reproducible a grafo dirigido; NetworkX se usa como oráculo.
Pruebas	pytest, unittest, GoogleTest	**pytest**	Parametrización, fixtures, aserciones y compatibilidad directa con el código Python.
Interfaz web	React + MapLibre, Dash, Flask + HTML	**React + FastAPI + MapLibre**	Separa la interfaz TypeScript del núcleo Python, expone contratos HTTP verificables y permite mapas interactivos sin acoplar el cálculo a la vista.
Aplicación móvil	Flutter/Dart, React Native, Kivy	**Flutter 3.47 + Dart 3.13**	Comparte el mismo contrato FastAPI sin duplicar solvers; ofrece interfaz táctil adaptativa, SQLite interno del dispositivo y compilación Android verificada.
Documentación	LaTeX, Typst, Word	**LaTeX + BibLaTeX/Biber**	Ecuaciones, tablas, referencias APA y control de versiones.

3.2. Criterios de Comparación

Se usaron cuatro criterios ponderados: C1 expresividad algorítmica (30 %), C2 instrumentación y aislamiento del ruido (25 %), C3 reproducibilidad y portabilidad (25 %) y C4 curva de aprendizaje (20 %). La Tabla 3 presenta una escala ordinal de 1 a 5. Los puntajes son un juicio técnico del equipo, no una medición física; se incluyen para hacer explícita la decisión, no para simular precisión inexistente.

Tabla 3
Matriz multicriterio del lenguaje principal (escala 1–5)

Alternativa	C1 (30 %)	C2 (25 %)	C3 (25 %)	C4 (20 %)	Ponderado
Python 3.12	5.0	4.2	4.8	4.8	**4.71**
C++20	3.8	4.8	3.5	2.8	3.78
Rust 1.75	4.0	4.6	3.6	2.2	3.69

3.3. Tecnología Seleccionada y Justificación

1. **Python 3.12.8.** Permite expresar los algoritmos con tipado y pruebas sin una cadena de compilación. Las tres colas se implementaron desde cero; no se usó `heapq` para las estructuras evaluadas.

La contrapartida es que los tiempos absolutos incluyen la sobrecarga del intérprete y no son extrapolables a C++ o Rust.

2. **OSMnx 2.1.1 y NetworkX 3.7.** OSMnx descarga y convierte la red vial; NetworkX se usa como oráculo de distancias. La red se congela en JSON para que el análisis no dependa de una descarga en vivo.

3. **pytest 9.1.1.** Pruebas parametrizadas, fixtures y ejecución selectiva. La suite completa se ejecuta con `python -m pytest python/tests -q`.

4. **React 19.2, TypeScript 5.9, Vite 7.2, MapLibre GL 6.16 y FastAPI 0.115.** Constituyen la plataforma web del proyecto: una aplicación TypeScript publicada en GitHub Pages para consultar rutas, comparar resultados, revisar la cobertura OSM y consultar la procedencia de los datos, conectada a una API Python desplegada en Render mediante Docker que reutiliza el núcleo. Dash y Flask + HTML se consideraron alternativas, pero no se implementaron. La interfaz es un medio de interacción; el núcleo evaluado sigue siendo el algoritmo.

5. **Flutter 3.47 y Dart 3.13 (aplicación móvil integrada).** Se implementó el cliente móvil multiplataforma en `mobile/` bajo arquitectura MVVM (`provider`, `go_router`, `flutter_map` y `sqflite`). Consume exactamente el mismo backend FastAPI (`v1.5.0`) para los algoritmos oficiales y la generación por semillas, evitando divergencias entre plataformas, y almacena borradores, instancias y resultados en la base SQLite interna del teléfono (sin servidor de base de datos externo). Android se compiló y verificó mediante APK de desarrollo; la compilación iOS queda estructurada para ejecutarse en un entorno macOS con Xcode.

6. **LaTeX con BibLaTeX y Biber.** Informe reproducible, ecuaciones y referencias APA. La compilación se automatiza con `compilar.ps1`.

7. **Servicios meteorológicos (Open-Meteo y OpenWeatherMap API).** Se implementó un diseño de doble proveedor en `src/services/weather.py`. Como opción sin credenciales se emplea Open-Meteo (datos de modelo). Como proveedor alternativo autenticado se integró `OpenWeatherMapClient` (https://home.openweathermap.org/users/sign_up). Ambas fuentes alimentan `WeatherObservation`; las consultas opcionales de la web son temporales y no reescriben el caso canónico. Una clave debe suministrarse localmente, nunca en archivos versionados.

8. **Modelo horario de tráfico.** Reglas deterministas y coeficientes hipotéticos por franja y tipo de vía; la interfaz muestra los factores, pero aplica solo el de avenida de forma uniforme a la ruta OSM. El cliente opcional de TomTom no dispone de un experimento canónico ni de pruebas de red en este repositorio; no se declara tráfico calibrado.

3.4. Procedimiento de Aprendizaje e Integración

El historial registra primero el prototipo con heap binario (02/10), luego las otras colas (04/10) y, el 28/10, los datos OSM y el banco completo. El piloto reducido se guardó el 04/11, después de esas mediciones. Estas fechas son de commits y archivos; no documentan cuándo estudió cada integrante ni acreditan sesiones de seguimiento.

3.5. Dificultades de Adopción y Soluciones Aplicadas

- **Dependencias geoespaciales en Windows.** OSMnx depende de bibliotecas geoespaciales cuya instalación puede variar entre equipos. Las pruebas de carga usan el *snapshot* JSON, sin repetir la descarga. `requirements.txt` fija versiones mínimas, no un entorno idéntico al de medición; falta comprobar una instalación desde cero.
- **Codificación de consola.** La consola CP1252 producía errores con caracteres Unicode. Se forzó UTF-8 en archivos y se usaron mensajes ASCII en los registros de la CLI.
- **Reproducibilidad de tiempos.** El banco guardado usa `perf_counter_ns`, calentamientos y mediana/IQR, con el recolector desactivado durante cada muestra. No se asume que una repetición produzca los mismos tiempos de pared.
- **Interfaz y estado.** React mantiene el estado de la sesión en el cliente; FastAPI valida cada solicitud y lee los snapshots versionados sin reescribir los JSON canónicos. Las consultas meteorológicas requieren una acción explícita y no se cachean como si fueran observaciones nuevas. La API no conserva sesiones de usuario ni resultados de ruta en el servidor.
- **Consulta meteorológica opcional.** Se integraron Open-Meteo y OpenWeatherMap. Cuando una clave falla o no hay red, se debe identificar como tal la respuesta guardada, sin presentarla como observación nueva. No se midieron tiempos de activación de claves.

4. Formulación del Problema

4.1. Definición Formal del Grafo Vial Urbano

El problema de cálculo de rutas mínimas en la red vial urbana del Cusco se modela formalmente como la determinación de caminos de costo acumulado mínimo sobre un grafo dirigido y ponderado:

$$G = (V, E, w) \tag{9}$$

donde:

- V es el conjunto finito de intersecciones viales, nodos semaforizados o cambios de geometría del trazado urbano.
- $E \subseteq V \times V$ es el conjunto de tramos viales dirigidos orientados según el sentido reglamentario de circulación vehicular. Cada par ordenado $(u, v) \in E$ indica que está permitido transitar desde la intersección u hacia la intersección v.
- $w : E \to \mathbb{R}_{\geq 0}$ es la función de costo aplicado o impedancia generalizada de viaje asignada a cada arista, cumpliendo estrictamente con la restricción de no negatividad:

$$w(u, v) \geq 0, \quad \forall (u, v) \in E \tag{10}$$

4.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema

Para un origen designado $s \in V$ y un destino $t \in V$, el problema de camino mínimo consiste en hallar

$$\delta(s,t) = \min_{P \in \mathscr{P}(s,t)} \sum_{e \in P} w(e), \tag{11}$$

donde $\mathscr{P}(s,t)$ es el conjunto de todos los caminos dirigidos simples que conectan s con t.

4.2.1. Variables de Decisión Binarias

Bajo el enfoque de optimización en redes, se define para cada arista dirigida $(u, v) \in E$ una variable de flujo binaria:

$$x_{uv} = \begin{cases} 1, & \text{si el tramo vial } (u, v) \text{ forma parte del camino mínimo óptimo;} \\ 0, & \text{en caso contrario.} \end{cases} \tag{12}$$

4.2.2. Salidas del Sistema

El sistema calcula dos estructuras fundamentales:

1. El vector de distancias mínimas definitivas $d : V \to \mathbb{R}_{\geq 0} \cup \{\infty\}$, donde $d[v] = \delta(s, v)$ representa el costo óptimo desde s hasta v.

2. El vector de predecesores $\pi : V \to V \cup \{\text{NIL}\}$, que permite reconstruir el camino encontrado por el algoritmo; pueden existir otros caminos con el mismo costo:

$$P(s,t) = \langle s = v_0, v_1, v_2, \ldots, v_k = t \rangle \tag{13}$$

4.3. Restricciones de Conservación de Flujo y No Negatividad

El flujo unitario inyectado en el origen s y absorbido en el destino t debe satisfacer el principio de balance de conservación de flujo en cada vértice $u \in V$:

$$\sum_{v:(u,v)\in E} x_{uv} - \sum_{v:(v,u)\in E} x_{vu} = \begin{cases} 1, & u = s \text{ (fuente)}, \\ -1, & u = t \text{ (sumidero)}, \\ 0, & u \in V \setminus \{s,t\} \text{ (nodos de tránsito intermedio)}, \end{cases} \tag{14}$$

sujeto a las restricciones de integridad y no negatividad de costos:

$$x_{uv} \in \{0,1\}, \quad w(u,v) \geq 0, \quad \forall (u,v) \in E \tag{15}$$

La no negatividad de w permite aplicar Dijkstra. Con costos cero puede haber varios caminos óptimos; basta escoger un camino simple.

4.4. Función Objetivo y Modelo de Costo con Impedancia Andina

La función objetivo minimiza el costo total acumulado del recorrido:

$$\min Z = \sum_{(u,v)\in E} w(u,v)\, x_{uv} \tag{16}$$

4.4.1. Formulación de la Impedancia Vial Andina

Se usan tres instancias distintas: la red didáctica de seis nodos, con longitudes y pendientes supuestas; el banco sintético, con pesos generados sin unidad física; y el recorte OSM, con longitudes y tiempos nominales guardados. La siguiente fórmula se aplica a los tiempos del ejemplo didáctico. No se aplica a los pesos sintéticos ni a la consulta OSM, que no incorpora pendientes por arista:

$$w(u,v) = \frac{L(u,v)}{v_{\text{nom}}(\tau(u,v))} \cdot \Big(1 + \alpha \max\big(0, \theta(u,v)\big)\Big) \cdot m_{\text{clima}} \cdot c_{\text{tráfico}}(\tau(u,v)), \tag{17}$$

donde:

- $L(u,v)$ es la longitud física del segmento vial en metros.
- $\tau(u,v)$ representa la tipología de la vía (avenida, calle centro, calle residencial, peatonal restringida).
- $v_{\text{nom}}(\tau)$ es la velocidad supuesta por tipo de vía para el ejemplo didáctico (Tabla 4); no equivale a un límite legal ni a velocidad observada.

- $\theta(u,v)$ es la pendiente longitudinal expresada como fracción decimal (por ejemplo, 0.03 para una gradiente del 3 %).
- $\alpha = 2.5$ es un coeficiente supuesto para el ejemplo; máx$(0,\theta)$ deja sin penalización las pendientes no positivas.
- $m_{\text{clima}} \geq 1$ es el factor de sensibilidad meteorológica.
- $c_{\text{tráfico}}(\tau) \geq 1$ es el coeficiente de congestión vehicular.

Tabla 4
Velocidades supuestas para la instancia didáctica; no usadas en la consulta OSM

Tipo de vía	Velocidad (km/h)	Velocidad (m/s)
Avenida	35	9.72
Calle residencial	25	6.94
Calle de centro histórico	15	4.17
Peatonal restringida	8	2.22

4.4.2. Variables Meteorológicas

El multiplicador m_{clima} procesa variables atmosféricas mediante el modelo de Open-Meteo consultado para las coordenadas del Cusco $(-13.5165, -71.9788)$:

$$m_{\text{clima}} = 1 + c_p \,\text{mín}\left(\frac{P}{P_{\text{ref}}}, 1\right) + c_h \,\text{máx}\left(0, \frac{H - H_0}{100 - H_0}\right) + c_v \left(1 - \text{mín}\left(\frac{V}{V_{\text{ref}}}, 1\right)\right) + c_w \,\text{mín}\left(\frac{W}{W_{\text{ref}}}, 1\right), \tag{18}$$

donde P es la precipitación en mm por intervalo ($P_{\text{ref}} = 10$ mm, $c_p = 0.20$), H es la humedad relativa en % ($H_0 = 65\,\%$, $c_h = 0.08$), V es la visibilidad en metros ($V_{\text{ref}} = 10\,000$ m, $c_v = 0.15$) y W es la ráfaga en km/h ($W_{\text{ref}} = 80$ km/h, $c_w = 0.10$). Los coeficientes son supuestos de sensibilidad, no calibración vial. La respuesta de modelo guardada en `applied_results.json` indica 28/10/2026 a las 12:30 (hora de Lima), $P = 0.1$ mm, $V = 8920$ m y $W = 40.7$ km/h. No contiene humedad medida: el cargador usa 50 % por defecto, sin efecto sobre el factor resultante $m_{\text{clima}} = 1.0691$. El valor proviene de un punto de grilla de Open-Meteo, no de una estación en la ciudad.

4.4.3. Modelo de Congestión Horaria Hipotético

El servicio `traffic.py` formula multiplicadores por franja y tipo de vía según supuestos analíticos:

$$c_{\text{tráfico}}(\tau) = 1 + \big(c_{\text{base}}(t) - 1\big) \cdot s(\tau) \cdot \gamma_{\text{sinergia}}, \tag{19}$$

donde $c_{\text{base}}(t) \in \{1.00, 1.25, 1.65, 2.20\}$ representa niveles horarios supuestos; $s(\tau)$ vale 1.35 para calle céntrica, 1.15 para avenida y 0.95 para residencial; y $\gamma_{\text{sinergia}} = 1 + 0.20\,\text{máx}(0, m_{\text{clima}} - 1)$. La consulta OSM guardada usa solo `travel_time_sec` y sensibilidad climática uniforme: no incluye un resultado experimental de congestión.

4.4.4. Supuestos del Modelo

Las semillas y los archivos locales fijan los pesos de cada instancia, pero no sus tiempos de cómputo. Las velocidades de la tabla y el coeficiente de pendiente son supuestos didácticos. En OSM, los tiempos base proceden del *snapshot*; el factor climático uniforme cambia el costo calculado sin cambiar el orden de los caminos. No se evaluó tráfico por tramo en esa consulta.

4.5. Ejemplo Ilustrativo Resuelto Analíticamente (Red Nuclear de 6 Nodos)

Para validar analíticamente el comportamiento del algoritmo, se diseñó la instancia representativa del Centro Histórico del Cusco mostrada en la Figura 1 y formalizada en la Tabla 5, con $|V| = 6$ intersecciones y $|E| = 9$ tramos viales dirigidos.

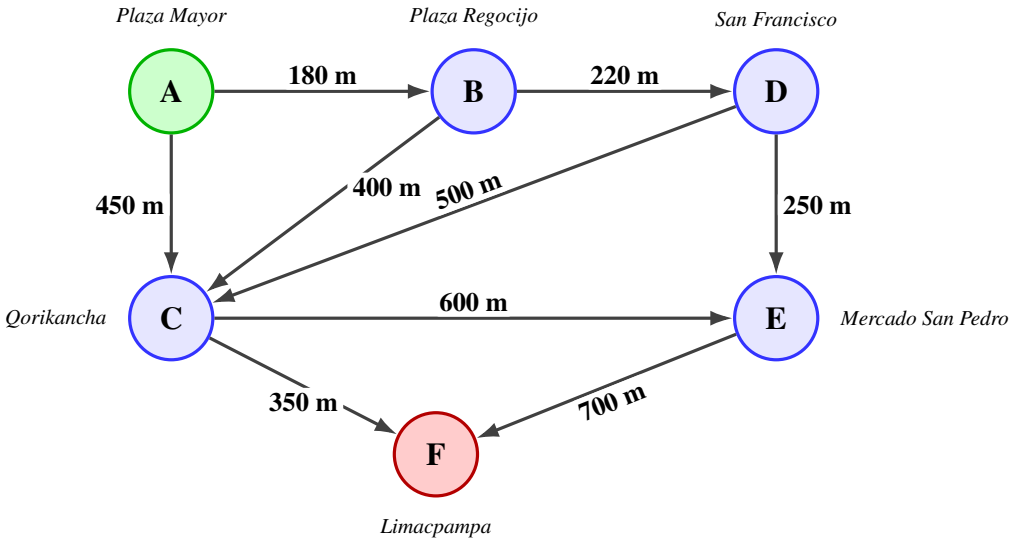


Figura 1
Instancia didáctica del Centro Histórico de Cusco ($|V| = 6$, $|E| = 9$). Longitudes esquemáticas en metros.

La Tabla 5 y la Figura 1 corresponden a una instancia didáctica. Usamos nombres de calles y lugares reales para facilitar la interpretación, pero no afirmamos que sus longitudes, pendientes o tiempos sean mediciones cartográficas. La red vial real empleada para los resultados aplicados se obtiene por separado desde OpenStreetMap mediante OSMnx.

Tabla 5
Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco.

Origen	Destino	Vía Urbana Asociada	Distancia (m)	Pendiente (%)	Tiempo Est. (s)
A	B	Calle Espaderos / Portal Carnicerías	180	+1.5	44.8
A	C	Calle Loreto / Pampa del Castillo	450	-2.0	108.0
B	D	Calle Garcilaso / Calle Marqués	220	+3.0	56.8
B	C	Calle San Bernardo / Almagro	400	-1.0	96.0
C	F	Calle Ahuacpinta / Limacpampa Grande	350	+0.5	36.5
C	E	Calle Matará / Tres Cruces de Oro	600	+2.0	151.2
D	E	Calle Santa Clara / Arco Santa Clara	250	+2.5	63.8
D	C	Calle Marqués / Mantas	500	-2.5	120.0
E	F	Calle Nueva Baja / Av. Tullumayo	700	-1.0	72.0

Nota. Los nombres de las vías corresponden a lugares reales del Cusco; la topología, las longitudes, las pendientes y los tiempos son valores esquemáticos definidos para la instancia didáctica. El tiempo se calcula con la función de costo e impedancia por pendiente andina ($\alpha = 2.5$), no con una medición de tráfico.

4.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$)

A continuación, se desarrolla analíticamente la traza paso a paso desde el origen $s = \mathrm{A}$ (Plaza Mayor) bajo la métrica de distancia física para verificación determinista:

- **Inicialización ($k = 0$):** $d[\mathrm{A}] = 0$, $d[v] = \infty$ para todo $v \neq \mathrm{A}$, $\pi[v] = \mathrm{NIL}$, $S = \emptyset$. La cola contiene todos los vértices: $Q = \{(\mathrm{A},0),(\mathrm{B},\infty),(\mathrm{C},\infty),(\mathrm{D},\infty),(\mathrm{E},\infty),(\mathrm{F},\infty)\}$.
- **Paso 1 ($k = 1$):** Se extrae $u = \mathrm{A}$ ($d[\mathrm{A}] = 0$). Vértice permanente: $S = \{\mathrm{A}\}$. Relajación de aristas salientes:
  - (A,B): $d[\mathrm{A}] + 180 = 180 < d[\mathrm{B}](\infty) \implies d[\mathrm{B}] \leftarrow 180$, $\pi[\mathrm{B}] \leftarrow \mathrm{A}$, `Decrease-Key`$(Q,\mathrm{B},180)$.
  - (A,C): $d[\mathrm{A}] + 450 = 450 < d[\mathrm{C}](\infty) \implies d[\mathrm{C}] \leftarrow 450$, $\pi[\mathrm{C}] \leftarrow \mathrm{A}$, `Decrease-Key`$(Q,\mathrm{C},450)$.
- **Paso 2 ($k = 2$):** Se extrae $u = \mathrm{B}$ ($d[\mathrm{B}] = 180$). $S = \{\mathrm{A},\mathrm{B}\}$. Relajaciones:
  - (B,D): $180 + 220 = 400 < d[\mathrm{D}](\infty) \implies d[\mathrm{D}] \leftarrow 400$, $\pi[\mathrm{D}] \leftarrow \mathrm{B}$, `Decrease-Key`$(Q,\mathrm{D},400)$.
  - (B,C): $180 + 400 = 580 \not< d[\mathrm{C}](450)$. Se descarta sin cambio.
- **Paso 3 ($k = 3$):** El mínimo en Q es $u = \mathrm{D}$ ($d[\mathrm{D}] = 400$). $S = \{\mathrm{A},\mathrm{B},\mathrm{D}\}$. Relajaciones:
  - (D,E): $400 + 250 = 650 < d[\mathrm{E}](\infty) \implies d[\mathrm{E}] \leftarrow 650$, $\pi[\mathrm{E}] \leftarrow \mathrm{D}$, `Decrease-Key`$(Q,\mathrm{E},650)$.
  - (D,C): $400 + 500 = 900 \not< d[\mathrm{C}](450)$. Se descarta.
- **Paso 4 ($k = 4$):** Se extrae $u = \mathrm{C}$ ($d[\mathrm{C}] = 450$). $S = \{\mathrm{A},\mathrm{B},\mathrm{D},\mathrm{C}\}$. Relajaciones:
  - (C,F): $450 + 350 = 800 < d[\mathrm{F}](\infty) \implies d[\mathrm{F}] \leftarrow 800$, $\pi[\mathrm{F}] \leftarrow \mathrm{C}$, `Decrease-Key`$(Q,\mathrm{F},800)$.
  - (C,E): $450 + 600 = 1050 \not< d[\mathrm{E}](650)$. Se descarta.

- **Paso 5 ($k = 5$):** Se extrae $u = \mathrm{E}$ ($d[\mathrm{E}] = 650$). $S = \{\mathrm{A}, \mathrm{B}, \mathrm{D}, \mathrm{C}, \mathrm{E}\}$. Relajaciones:
  - (E, F): $650 + 700 = 1350 \not< d[\mathrm{F}](800)$. Se descarta.
- **Paso 6 ($k = 6$):** Se extrae el último vértice $u = \mathrm{F}$ ($d[\mathrm{F}] = 800$). $S = V$. La cola Q queda vacía. Finaliza la ejecución.

Tabla 6
Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo.

Iter.	Vértice u	Acción / Relajación	A	B	C	D	E	F
$k = 0$	Inicial	Inserción en cola	0 / -	∞ / -	∞ / -	∞ / -	∞ / -	∞ / -
$k = 1$	A ($d = 0$)	Relaja B (180), Relaja C (450)	**0** / -	180 / A	450 / A	∞ / -	∞ / -	∞ / -
$k = 2$	B ($d = 180$)	Relaja D (400), Descarta C	**0** / -	**180** / A	450 / A	400 / B	∞ / -	∞ / -
$k = 3$	D ($d = 400$)	Relaja E (650), Descarta C	**0** / -	**180** / A	450 / A	**400** / B	650 / D	∞ / -
$k = 4$	C ($d = 450$)	Relaja F (800), Descarta E	**0** / -	**180** / A	**450** / A	**400** / B	650 / D	800 / C
$k = 5$	E ($d = 650$)	Descarta F	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	800 / C
$k = 6$	F ($d = 800$)	Sin aristas salientes	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	**800** / C

Nota. Notación *distancia / predecesor* ($d[v]/\pi[v]$). Cifras en negrita indican nodos cerrados en S.

Con el vector de predecesores $\pi = \{\mathrm{A} : \mathrm{NIL}, \mathrm{B} : \mathrm{A}, \mathrm{C} : \mathrm{A}, \mathrm{D} : \mathrm{B}, \mathrm{E} : \mathrm{D}, \mathrm{F} : \mathrm{C}\}$ se reconstruyen estas rutas mínimas desde A:

$$\text{Ruta hacia Mercado San Pedro (E):} \quad \mathrm{A} \to \mathrm{B} \to \mathrm{D} \to \mathrm{E} \quad (180 + 220 + 250 = 650.0 \text{ m}), \tag{20}$$

$$\text{Ruta hacia Plazoleta Limacpampa (F):} \quad \mathrm{A} \to \mathrm{C} \to \mathrm{F} \quad (450 + 350 = 800.0 \text{ m}). \tag{21}$$

Bajo la métrica de tiempo efectivo con impedancia topográfica (Tabla 5), los costos son 165.33 s y 144.45 s, respectivamente, coincidiendo exactamente con la salida del software (Tabla 7).

Tabla 7
Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor

Nodo	Nombre de la Intersección	Distancia Pura (m)	Tiempo Efectivo (s)	Ruta óptima
A	Plaza Mayor del Cusco	0.0	0.0	`A`
B	Plaza Regocijo (Cusipata)	180.0	44.8	`A ->B`
C	Qorikancha (Santo Domingo)	450.0	108.0	`A ->C`
D	Plaza San Francisco	400.0	101.6	`A ->B ->D`
E	Mercado Central San Pedro	650.0	165.3	`A ->B ->D ->E`
F	Plazoleta Limacpampa	800.0	144.4	`A ->C ->F`

Nota. Solución calculada desde el vértice fuente $s = \mathrm{A}$ (Plaza Mayor).

4.7. Instancia de verificación canónica de CLRS

Para comprobar la adhesión a la literatura clásica, el sistema incluye la instancia de caminos mínimos presentada por Cormen et al. (2022, figura 24.6, p. 659). El orden de extracción obtenido por el software es $\langle s, y, z, t, x \rangle$, con distancias definitivas $\langle 0, 5, 7, 8, 9 \rangle$, idénticas a la solución teórica.

4.8. Red Vial Urbana Real de Cusco (OpenStreetMap)

El recorte OSM de conducción se descargó con OSMnx 2.1.1 alrededor de $(-13.5165, -71.9788)$, en un radio de 1,500 m. El *snapshot* `cusco_osm_drive.json` guarda 891 nodos y 2,139 aristas dirigidas (2.40 aristas por vértice). El origen de la consulta es el nodo más próximo al centro indicado, no el centroide del grafo; el destino es el nodo alcanzable de mayor costo base desde ese origen. La ruta calculada contiene 17 vértices, suma 2,810.6 m y tiene un costo estimado de viaje de 387.197 s. Este costo no es el tiempo de cómputo de Dijkstra.

5. Diseño Algorítmico

5.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta

El Algoritmo de Dijkstra es el método canónico para la resolución del problema de caminos mínimos desde un origen único en grafos con pesos no negativos. Su estructura lógica opera sobre una cola de prioridad genérica Q. El procedimiento detallado se formaliza en el Algoritmo 1.

Algorithm 1 Algoritmo de Dijkstra con cola de prioridad parametrizada

Require: Grafo dirigido $G = (V, E, w)$, Vértice origen $s \in V$, Cola de prioridad parametrizada Q
Ensure: Vector de distancias mínimas $d : V \to \mathbb{R}_{\geq 0} \cup \{\infty\}$, Predecesores $\pi : V \to V \cup \{\text{NIL}\}$

// [L1-L6] Inicialización del estado algorítmico
for all $v \in V$ **do**
  $d[v] \leftarrow \infty$
  $\pi[v] \leftarrow$ NIL
end for
$d[s] \leftarrow 0.0$
for all $v \in V$ **do**
  Q.insert$(v, d[v])$ {La implementación inserta todos los vértices}
end for
$S \leftarrow \emptyset$ {Conjunto de vértices con distancia permanente cerrada}
while not Q.is_empty() **do**
  $(d_u, u) \leftarrow Q$.extract_min() {// [L9-L10] Extracción del vértice con menor distancia tentativa}
  if $d_u = \infty$ **then**
    break {// [L11-L13] Vértices restantes inalcanzables desde s}
  end if
  $S \leftarrow S \cup \{u\}$ {// [L14] Incorporación de u al conjunto permanente}
  for all $(u, v) \in E$ **do**
    if $v \notin S$ **and** $d_u + w(u, v) < d[v]$ **then**
      $d[v] \leftarrow d_u + w(u, v)$ {// [L15-L18] Relajación voraz de la arista}
      $\pi[v] \leftarrow u$
      Q.decrease_key$(v, d[v])$ {Disminución de prioridad en Q}
    end if
  end for
end while
return (d, π)

5.1.1. Correspondencia entre Pseudocódigo e Implementación

La Tabla 8 relaciona el pseudocódigo con `python/src/algorithms/dijkstra.py`. El solver también acepta una parada opcional al llegar al destino y puede reconstruir caminos después del intervalo principal.

Tabla 8
Correspondencia entre pseudocódigo y la implementación en Python

Etapa	**Pseudocódigo**	**Código Python** (`dijkstra.py`)
Inicialización	$d[v] \leftarrow \infty$, $\pi[v] \leftarrow$ NIL, $d[s] \leftarrow 0$	Diccionarios `dist`, `prev` y asignación `dist[s]=0.0`.
Construcción de Q	Insertar cada $v \in V$ con clave $d[v]$	`create_queue(queue_type)` y `heap.insert` para cada nodo.
Extracción del mínimo	$(d_u, u) \leftarrow Q$.extract_min()	Invocación polimórfica `d_u, u = heap.extract_min()`.
Criterio de corte	**if** $d_u = \infty$ **then break**	`if d_u == INF: break` (componentes disjuntas).
Relajación voraz	$d_u + w(u,v) < d[v]$	`candidate_d = d_u + weight`; comparar con `dist[v]`.
Actualización de cola	`decrease_key`	`heap.decrease_key(v, candidate_d)` para un nodo no visitado.
Reconstrucción	Retorno de (d, π) y traza	Módulo `reconstruct_path(prev, s, t)` y telemetría de operaciones.

5.2. Invariantes de Bucle y Demostración Formal de Corrección

La corrección formal del algoritmo descansa sobre el siguiente invariante inductivo:

5.2.1. Invariante de Bucle Fundamental

Al inicio de cada iteración del ciclo `while`:

1. Para todo vértice permanente $u \in S$, el valor estimado coincide con la distancia mínima real: $d[u] = \delta(s,u)$.
2. Para $v \in Q = V \setminus S$, $d[v]$ es el menor costo descubierto hasta v mediante aristas salientes de vértices procesados en S; vale ∞ si aún no se descubrió un camino. El origen inicia con $d[s] = 0$.

5.2.2. Demostración por Inducción Matemática de Optimalidad

- **Base ($|S| = 0$):** Al comenzar, $d[s] = 0$ y las otras distancias valen ∞. Aún no se han procesado aristas salientes; la segunda propiedad describe precisamente el estado descubierto hasta entonces.
- **Paso inductivo:** Supóngase que el invariante se satisface para $|S| = k$. El algoritmo selecciona el vértice $u \in Q$ con $d_u = \min_{z \in Q} d[z]$ y lo incorpora a S. Si existiese un camino dirigido P desde s hasta u estrictamente más corto que $d[u]$ ($w(P) < d[u]$), dicho camino debe salir de S hacia Q por primera vez a través de alguna arista (x,y), con $x \in S$ y $y \in Q$. La longitud acumulada hasta y satisface:

$$w(P) \geq \delta(s,x) + w(x,y) \geq d[y] \tag{22}$$

porque al procesar x se consideró (x,y), y la continuación de P desde y tiene peso no negativo. Como u tiene la menor clave pendiente, $d[y] \geq d[u]$, contradiciendo $w(P) < d[u]$. Por tanto, $d[u] = \delta(s,u)$. Relajar las aristas salientes de u actualiza las claves descubiertas para la siguiente iteración.

- **Terminación:** Cuando $Q = \emptyset$ o los vértices restantes tienen distancia ∞, S contiene todos los nodos alcanzables desde s, garantizando que $d[v] = \delta(s,v)$ y que el árbol de predecesores π codifica las rutas óptimas.

5.3. Estructuras de Datos de Cola de Prioridad Evaluadas

5.3.1. Min-Heap Binario Indexado (`IndexedMinHeap`)

El montículo binario indexado se representa sobre un arreglo contiguo en memoria. Para posibilitar la operación `decrease_key` en tiempo sublineal sin requerir búsquedas lineales exhaustivas, se implementa una tabla hash inversa de posiciones $\text{pos} : V \to \mathbb{N}$. Cada movimiento de elementos durante el flotado (`sift-up`) o hundido (`sift-down`) ejecuta un intercambio atómico:

$$\text{heap}[i] \leftrightarrow \text{heap}[j], \quad \text{pos}[\text{heap}[i].\text{id}] \leftarrow i, \quad \text{pos}[\text{heap}[j].\text{id}] \leftarrow j \tag{23}$$

Con este esquema, localizar cualquier vértice toma $O(1)$ y el flotado recorre a lo sumo $\lfloor \log_2 |V| \rfloor$ niveles, logrando una complejidad estricta de $O(\log |V|)$ por decremento con un consumo de memoria lineal $O(|V|)$.

5.3.2. 4-ary Heap Contiguo (`DAryHeap`, $d = 4$)

El montículo 4-ario generaliza el heap binario adoptando un factor de ramificación $d = 4$. La navegación entre niveles se realiza mediante aritmética entera directa sin punteros:

$$\text{padre}(i) = \left\lfloor \frac{i-1}{4} \right\rfloor, \quad \text{hijo}_k(i) = 4i + k, \quad k \in \{1,2,3,4\} \tag{24}$$

Esta formulación reduce aproximadamente a la mitad la profundidad del árbol: $\log_4 |V| = \frac{1}{2} \log_2 |V|$ (al redondear al entero superior, la igualdad exacta no siempre se conserva). `decrease_key` puede recorrer menos niveles; `extract_min` debe comparar hasta cuatro hijos por nivel. La disposición contigua es una propiedad del arreglo, pero no medimos los accesos a caché.

5.3.3. Fibonacci Heap (`FibonacciHeap`)

El Fibonacci Heap (Fredman & Tarjan, 1987) mantiene una colección de árboles con orden de montículo estructurados mediante listas circulares doblemente enlazadas con punteros `parent`, `child`, `left`, `right`, `degree` y el bit de marcado `marked`.

- `insert`: Inserta un nuevo árbol unitario en la lista de raíces en $O(1)$ peor caso.
- `decrease_key`: Si la clave relajada viola el orden de montículo con su padre, se corta el nodo hacia la lista de raíces en $O(1)$ amortizado. Si el padre ya estaba marcado, se propaga un corte en cascada recursivo (*cascading cut*) garantizando que el grado de cualquier nodo satisfaga $D(n) \leq \lfloor \log_\phi n \rfloor \approx 1.4404 \log_2 n$.

- `extract_min`: Elimina el nodo raíz mínimo, promueve todos sus hijos a la lista de raíces y ejecuta una consolidación sistemática uniendo árboles de igual grado mediante un arreglo dinámico dimensionado con la cota de la razón áurea $\lceil \log_{\phi} n \rceil + 2$.

5.4. Análisis Comparativo de Complejidades Teóricas

La Tabla 9 expone la comparación de cotas asintóticas temporales y espaciales entre las tres estructuras analizadas y la referencia elemental de arreglo no ordenado.

Tabla 9
Cotas temporales y espaciales de las colas de prioridad evaluadas

Estructura	Insert	Extract-Min	Decrease-Key	Espacio	Costo total en Dijkstra
Arreglo no ordenado	$O(1)$	$O(\|V\|)$	$O(1)$	$O(\|V\|)$	$O(\|V\|^2 + \|E\|)$
Heap binario	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\|V\|)$	$O((\|V\| + \|E\|) \log \|V\|)$
Heap 4-ario ($d = 4$)	$O(\log_4 \|V\|)$	$O(4 \log_4 \|V\|)$	$O(\log_4 \|V\|)$	$O(\|V\|)$	$O((\|E\| + 4\|V\|) \log_4 \|V\|)$
Fibonacci Heap	$O(1)$	$O(\log \|V\|)^*$	$O(1)^*$	$O(\|V\|)$	$O(\|E\| + \|V\| \log \|V\|)$

Nota. *Cota amortizada. Para d fijo, $\log_d |V| = O(\log |V|)$; la tabla no determina por sí sola los tiempos observados.

Las cotas temporales de la Tabla 9 consideran hasta $|V|$ inserciones y extracciones y hasta $|E|$ relajaciones: son cotas de peor caso para los montículos binario y 4-ario, y amortizadas para Fibonacci (una extracción aislada puede costar $O(|V|)$). No se asigna una cota de caso promedio sin definir una distribución de grafos y costos. La columna de espacio corresponde a la *cola*: Dijkstra requiere además $O(|V|)$ de memoria auxiliar para distancias, predecesores y visitados, sobre una entrada de $O(|V| + |E|)$. La implementación reconstruye por defecto la ruta a cada destino; almacenar y construir todas esas rutas puede añadir $O(|V|^2)$ espacio y tiempo en el peor caso. Las cotas temporales de la tabla describen el núcleo sin esa reconstrucción (`build_paths=False`); la traza opcional también incrementa la memoria utilizada.

5.5. Guía Práctica de Selección según el Escenario

La Tabla 10 sintetiza el marco de selección tecnológica deducido a partir de los fundamentos algorítmicos y el perfil de la red vial.

Tabla 10
Guía de selección práctica de algoritmos y estructuras en redes viales

Alternativa	Escenario Recomendado	Justificación Técnica y Evidencia
Dijkstra + Heap Binario	Opción inicial para esta implementación; contrastar con la red de interés.	Menor mediana en la consulta OSM de Cusco (15.65 ms) y en 3 de 10 escalas sintéticas.
Dijkstra + Heap 4-ario	Alternativa para repetir la comparación con otras escalas y grafos.	Menor mediana en 7 de 10 escalas sintéticas; los IQR frente al binario se solapan en todas.
Dijkstra + Fibonacci	Interés teórico por su cota amortizada; falta evaluar otras implementaciones.	En estas 10 escalas fue entre 1.05 y 1.59 veces más lento que el binario; no se midieron las causas de hardware.
A^* **con heurística consistente**	Consultas punto a punto con coordenadas y pesos compatibles.	En el par OSM probado expandió 7.2 % menos vértices con el mismo costo; no se comparó su tiempo con las tres colas.
Bellman-Ford	Oráculo de validación cruzada y verificación de ausencia de ciclos negativos.	Complejidad $O(\|V\|\|E\|)$; opera como juez independiente para validar la suite de pruebas unitarias.

6. Diseño e Implementación del Sistema

6.1. Arquitectura Modular del Sistema (4 Capas)

La Figura 2 agrupa módulos por función: datos, colas, algoritmos y pruebas/interfaz. Es un esquema de componentes, no un diagrama literal de llamadas: Dijkstra recibe directamente el grafo y usa la cola elegida. El cálculo no necesita la interfaz web ni una consulta de red para las instancias locales.

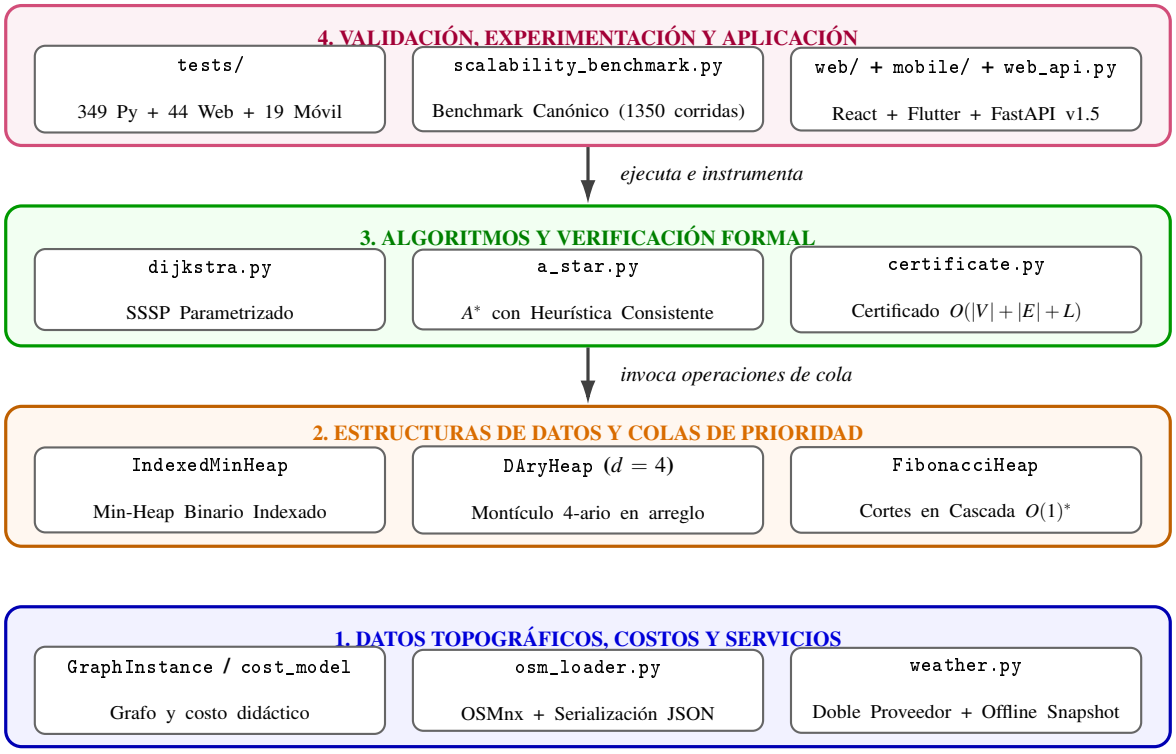


Figura 2
Agrupación funcional de módulos; las flechas superiores indican llamadas, no un flujo exclusivo de datos

6.2. Descripción Detallada de Módulos y Componentes

El software se estructura modularmente dentro del directorio `python/`:

1. `src/algorithms/dijkstra.py`: Motor algorítmico principal que implementa el algoritmo de Dijkstra con inyección de colas de prioridad y telemetría de operaciones elementales.

2. `src/algorithms/a_star.py`: Búsqueda heurística A^* con precomputación del factor admisible λ, construcción de heurística consistente euclidiana escalada y métricas de poda.

3. `src/algorithms/reference.py`: Oráculos de referencia independientes (Bellman-Ford y Dijkstra ingenuo sobre lista no ordenada).

4. `src/datastructures/`: Implementación pura de `IndexedMinHeap`, `DAryHeap` ($d = 4$) y `FibonacciHeap` bajo contrato polimórfico común.

5. `src/io/graph_io.py`: Lectura y validación de instancias JSON, TXT, CSV y aristas en línea.

6. `src/io/osm_loader.py`: Descarga y serialización de redes OpenStreetMap mediante OSMnx; el *snapshot* guarda coordenadas WGS84 y se puede cargar sin red.

7. `src/services/weather.py`: Servicio meteorológico desacoplado con arquitectura de doble proveedor (Open-Meteo REST API y OpenWeatherMap API v2.5), persistencia en *snapshots* y respaldo ante fallos de conexión.

8. `src/services/traffic.py`: Modelo de congestión vehicular horaria y coeficientes por clase de vía.

9. `src/verification/certificate.py`: Verifica distancias y predecesores en $O(|V|+|E|)$, más el recorrido de los caminos explícitos si se proporcionan; compara soluciones.

10. `src/benchmarks/scalability_benchmark.py`: Banco de experimentación sistemática para medir tiempos y primitivas en perfiles sintéticos dispersos y densos.

11. `python/web_api.py`, `web/` y `mobile/`: API FastAPI (`v1.5.0`) con modelos de respuesta explícitos, límites unificados en `/api/v1/catalog` y exportación completa en `POST /api/v1/instances/export`; interfaz web React/TypeScript (8 vistas) y cliente móvil Flutter/Dart (5 secciones adaptativas para teléfonos y tabletas) que no alteran los JSON canónicos.

12. `main.py` y `src/cli.py`: Interfaz de línea de comandos para automatización y demostración en vivo.

6.3. Contrato Unificado e Integración de la Aplicación Móvil (`mobile/`)

Para garantizar equivalencia exacta de costos, límites y resultados entre la interfaz web y la aplicación móvil sin duplicar la lógica algorítmica en Dart, ambos clientes consumen `python/web_api.py` (desplegada mediante Docker en Render con health check `/api/v1/health`). Las decisiones clave de esta integración son:

Contrato tipado y límites compartidos `GET /api/v1/version` separa versiones de API, datos, documentación y generador; `GET /api/v1/catalog` expone los límites de validación (hasta 5 000 vértices, 30 000 aristas, 2 MB de carga útil y tamaños experimentales de 5 a 200 nodos) compartidos entre FastAPI, React (`experimentInput.ts`) y Flutter.

Exportación sin recorte `POST /api/v1/instances/validate` limita `display_edges` a 3 000 aristas para fluidez visual, mientras que `POST /api/v1/instances/export` entrega la instancia completa (incluidos grafos densos generados por semilla e identificadores mixtos entero/cadena) para su reimportación sin pérdida.

Almacenamiento interno del dispositivo (sin servidor de BD externo) La aplicación Flutter utiliza una base SQLite local (`rutas_cusco_local.db` mediante `sqflite`, esquema versionado) para guardar borradores editables, instancias importadas o generadas (`saved_instances`), consultas calculadas (`saved_routes`) y experimentos de sesión (`saved_experiments`), además de `SharedPreferences` para ajustes del servidor y lecturas climáticas guardadas explícitamente.

Operación sin conexión y separación de tiempos Sin Internet, la aplicación permite estudiar 16 ejemplos incluidos, editar borradores, revisar el banco canónico (1 350 muestras), el piloto (18 muestras), la documentación técnica y resultados previamente guardados (rotulados como «Resultado guardado · sin conexión»). Los tiempos puros del solver en Python (μs/ms) se muestran separados de la latencia total de solicitud en red del dispositivo.

6.3.1. Arquitectura y Diseño de la Aplicación Móvil Flutter

El cliente móvil (`mobile/`) se implementó en Flutter 3.47 con Dart 3.13 bajo una arquitectura modular por capas orientada a mantenibilidad:

1. **Capa de red y datos (`core/network/`, `data/`):** Centralizada en `ApiClient` con control de concurrencia mediante `RequestGuard` para descartar respuestas de solicitudes desfasadas o canceladas, manejo de excepciones tipadas (`ApiException`) y persistencia local reactiva estructurada en repositorios desacoplados (`ServerRepository`, `InstanceRepository`, `RouteRepository`, `ConditionsRepository`, `AnalysisRepository`, `DocsRepository` y `SettingsRepository`).

2. **Gestión de estado y navegación adaptativa (`app/`):** Implementada con el patrón MVVM soportado en `ChangeNotifierProvider` y enrutamiento declarativo vía `go_router` (`StatefulShellRoute`). La interfaz conmuta automáticamente entre `NavigationBar` inferior en teléfonos compactos (< 640 px) y `NavigationRail` lateral en tabletas y pantallas amplias (≥ 640 px).

3. **Identidad visual institucional y componentes táctiles:** Adopta la paleta oficial de la UNSAAC (`#681C32` granate, `#8A6418` dorado oscuro, `#1F1B1C` tinta y `#F7F3EA` papel) definida en `DESIGN.md`. Los selectores desplegables estándar de escritorio se reemplazaron por componentes táctiles modales (`AppSelectField`) que despliegan hojas inferiores redondeadas (*bottom sheets*) con subtítulos explicativos e iconos distintivos, y controles segmentados (`SegmentedButton`) para la elección de algoritmo.

4. **Visualización responsive sin desbordes (`ResponsiveDataView`):** Para tablas de complejidad, benchmarks canónicos y registros piloto, el componente conmuta dinámicamente: en teléfonos genera tarjetas estructuradas legibles con cifras tabulares que evitan el desplazamiento horizontal incómodo, mientras que en pantallas anchas proyecta tablas formales de datos.

6.3.2. Módulos de la Aplicación Móvil

La aplicación organiza sus capacidades en cinco secciones principales accesibles desde la barra de navegación:

- **Rutas (Figura 3):** Permite seleccionar origen y destino mediante identificador de vértice, coordenadas GPS en tiempo real o pulsación directa sobre el mapa. El visualizador integra `flutter_map` con teselas OpenStreetMap o plano local abstracto, complementado con controles flotantes (píldora superior de selección Origen/Destino/Bloqueo para evitar modificaciones accidentales al navegar, y barra lateral con botones de acercar, alejar, encuadre de red, centrado de ruta y ampliación a

pantalla completa). Al pulsar «Calcular y contrastar», resuelve la ruta en el backend y presenta simultáneamente el costo óptimo, longitud, tiempo estimado, operaciones de cola, tiempo del solver y la comparación de las cuatro variantes algorítmicas (Dijkstra con colas Binaria, 4-aria, Fibonacci y A^*).

- **Instancias:** Ofrece el catálogo de 16 redes canónicas y didácticas, un editor táctil completo de nodos y aristas con validación local preliminar, duplicación en copias editables, importación de archivos JSON, CSV o TXT y generador reproducible oficial con semillas.
- **Análisis (Figura 4a):** Permite cotejar dos consultas guardadas en SQLite, consultar las gráficas de dispersión y medianas de las 1 350 muestras del banco canónico por cola y ejecutar nuevos experimentos controlados conectándose a FastAPI.
- **Biblioteca y Referencia Académica:** Administra instancias y rutas guardadas en el almacenamiento interno SQLite, incorpora el módulo de Referencia Académica con síntesis de complejidades asintóticas, modelo de costos de impedancia y visualizador/compartidor del informe final de investigación en formato PDF.
- **Ajustes y Acerca de (Figura 4b):** Monitoriza el estado de salud y latencia del servicio backend, ofrece diagnóstico y reintento de conexión, documentación de uso sin internet y de capas cartográficas, y contiene la sección institucional «Acerca de» con la versión instalada obtenida dinámicamente del paquete (`package_info_plus`), el alcance formativo del software y la acreditación explícita del equipo de desarrollo del Grupo 5 de la UNSAAC.



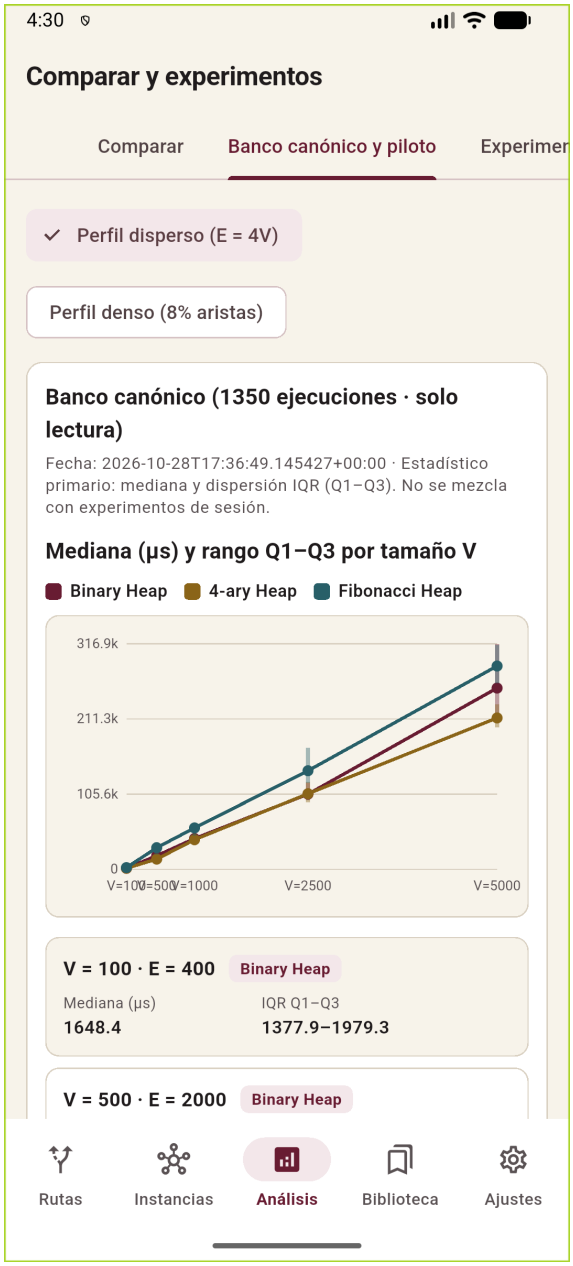
(a) *Mapa interactivo con ruta trazada y controles flotantes.*



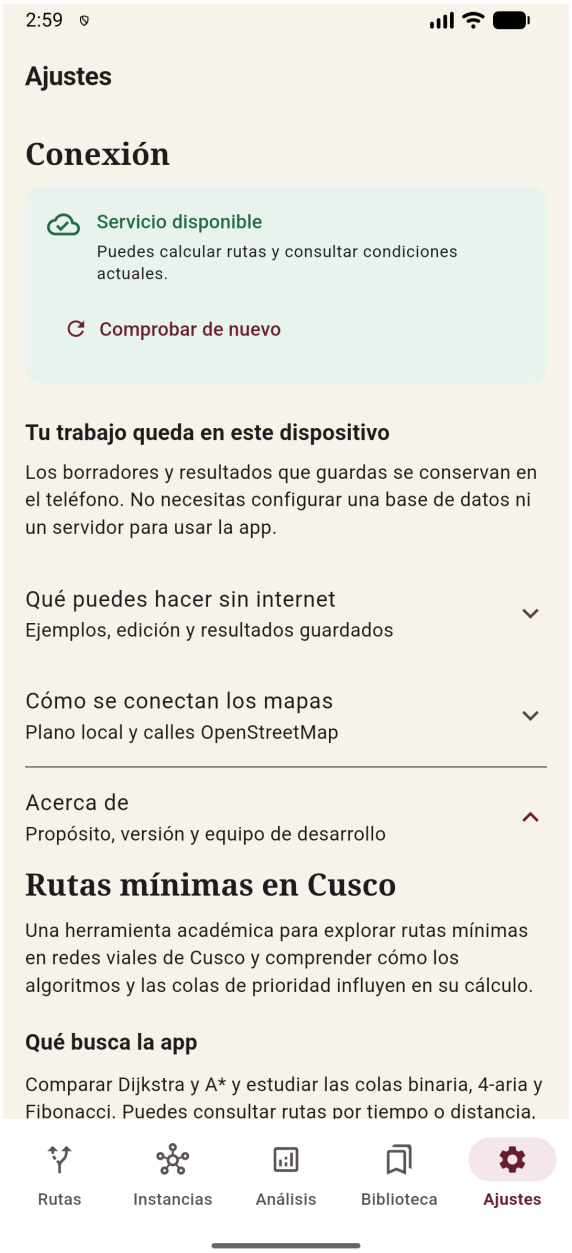
(b) *Métricas detalladas y comparación simultánea de 4 variantes.*

Figura 3

Módulo de cálculo de Rutas en la aplicación móvil Flutter (`mobile/`). A la izquierda, visualización de la ruta mínima calculada sobre el plano vial de Cusco con controles flotantes de interacción, zoom y bloqueo. A la derecha, desglose cuantitativo con distinción entre tiempo del solver en servidor y latencia de red, junto al contraste de las cuatro variantes algorítmicas.



(a) *Módulo de Análisis con gráfica de colas y tarjetas responsive.*



(b) *Pantalla de Ajustes y panel institucional «Acerca de».*

Figura 4

Módulos de Análisis y Ajustes en la aplicación móvil Flutter. A la izquierda, inspección visual de las medianas e IQR del banco canónico de 1 350 corridas. A la derecha, estado de conectividad en vivo con el backend FastAPI, guía de operación sin internet y panel «Acerca de» con versión instalada y autoría académica del Grupo 5 (UNSAAC).

6.4. Formatos y Estructura de Datos (Serialización JSON)

Los resultados y *snapshots* principales se guardan en JSON (Tabla 11); la CLI también acepta instancias TXT, CSV y en línea. Las pruebas que usan los archivos locales no requieren consultar APIs en vivo.

Tabla 11

Archivos de datos principales y su rol funcional en el sistema

Archivo en Repositorio	**Contenido y Propósito**
data/instances/cusco_centro.json	Instancia nuclear didáctica (6 nodos, 9 aristas, longitudes, pendientes y tiempos).
data/processed/cusco_osm_drive.json	Recorte OSM guardado (891 nodos con lat/lon WGS84, 2,139 aristas dirigidas); sin pendientes ni tráfico por tramo.
data/processed/cusco_urbano_drive.json	Snapshot OSM ampliado (1,938 nodos y 4,753 aristas) de una consulta radial de 3 km recortada por distancia de red; revisado y separado del caso canónico.
data/processed/benchmark_results.json	Protocolo experimental, ambiente, muestras cronometradas, medianas e IQR de las 10 escalas.
data/processed/applied_results.json	Mediciones de la consulta OSM, comparación de colas, poda de A^* y sensibilidad climática.
data/processed/weather_cusco.json	Respuesta meteorológica posterior; no es la observación usada en `applied_results.json`.

```
{
  "origin": "A",
  "destination": "B",
  "length_m": 180.0,
  "street_type": "calle_centro",
  "gradient_pct": 1.5,
  "street_name": "Calle Espaderos / Portal Carnicerias",
  "travel_time_sec": 44.82
}
```

Listing 1: Extracto del esquema JSON serializado para un segmento vial cusqueño

6.5. Contratos de Interfaz e Inyección de Colas de Prioridad

Para posibilitar la sustitución dinámica y transparente de colas en el solver de Dijkstra sin alterar el flujo algorítmico, cada una de las tres estructuras implementa una interfaz uniforme:

- `insert(item: str, key: float) ->None`: Inserta un identificador de vértice con su distancia tentativa asociada.
- `extract_min() ->Tuple[float, str]`: Extrae y retorna el par $(\text{clave}, \text{elemento})$ de menor valor acumulado, reestructurando el montículo según sus invariantes.
- `decrease_key(item: str, new_key: float) ->None`: Disminuye la clave del vértice a su nuevo valor relajado, flotando el nodo o ejecutando cortes en cascada.
- `is_empty() ->bool`: Determina en $O(1)$ si existen vértices pendientes de procesamiento.
- `op_stats() ->Dict[str, int]`: Retorna la telemetría acumulada de inserciones, extracciones, decrementos de clave y comparaciones.

6.6. Manejo Defensivo de Entradas y Validación

La validación centralizada en `src/io/graph_io.py` protege al sistema contra datos corruptos, rechazando de forma determinista: JSON mal formado, campos requeridos faltantes, identificadores duplicados, pesos no numéricos, NaN o infinitos, costos negativos (en Dijkstra) y nodos de origen/destino inexistentes. La CLI retorna códigos de salida estandarizados: 0 (éxito), 1 (fallo en la verificación del certificado de optimalidad) y 2 (entrada inválida o error de sintaxis).

6.7. Servicios Complementarios: Meteorología y Escenarios de Tráfico

El módulo `src/services/weather.py` implementa un desacoplamiento mediante una arquitectura de doble proveedor:

1. **Open-Meteo REST API (Open-Meteo, 2026):** Fuente pública libre sin autenticación basada en datos de modelos meteorológicos de predicción numérica.

2. **OpenWeatherMap API (v2.5 Current Weather) (OpenWeather, 2026):** Proveedor autenticado mediante clave de API suministrada exclusivamente por variables de entorno locales, con mapeo hacia `WeatherObservation`.

Ambos clientes alimentan la función multiplicadora $\Omega_{\text{clima}} \geq 1.0$:

$$\Omega_{\text{clima}} = 1.0 + c_{\text{lluvia}}\,\text{mín}\left(\frac{p}{10}, 1.0\right) + c_{\text{vis}}\,\text{máx}\left(0, 1 - \frac{v}{10\,000}\right) + c_{\text{ráfaga}}\,\text{mín}\left(\frac{r}{80}, 1.0\right) + c_{\text{hum}} f(\text{HR}) \tag{25}$$

El *snapshot* local permite usar un dato guardado sin conexión. No se presenta como lectura meteorológica nueva ni como respaldo universal ante cualquier fallo del proveedor.

6.8. Decisiones de Implementación Relevantes y Justificación Técnica

- **Colas propias:** Las tres colas se implementan en Python sin `heapq`. Esto permite comparar sus implementaciones aquí, no aislar por completo las constantes del intérprete.
- **Degeneración admisible de A^*:** Si un grafo no posee coordenadas geográficas, A^* adopta $h(v) = 0$, degenerando de manera admisible en el algoritmo canónico de Dijkstra sin romper la ejecución.
- **Datos locales:** La red y la observación climática del experimento se guardan en `data/processed/`. Las semillas reproducen las instancias sintéticas; los tiempos de ejecución varían según el equipo y la carga del sistema.
- **Separación de visualización:** El banco llama directamente al solver; el renderizado web no forma parte del intervalo cronometrado.

7. Estrategia de Pruebas

7.1. Framework de Pruebas y Patrón Arrange-Act-Assert (AAA)

La suite en Python con `pytest` contiene 349 casos, complementada por 44 pruebas de la interfaz web en Vitest y 40 pruebas unitarias y de widgets en Flutter (433 pruebas automatizadas en total al 13/11/2026). El patrón **Arrange–Act–Assert (AAA)** ayuda a distinguir preparación, ejecución y comprobación en los casos; no se verificó que cada uno de los 349 esté escrito con esa misma estructura. La Sección 2.1.5 explica el patrón:

- **Arrange (Organizar):** Configuración del entorno determinista, definición de semillas pseudoaleatorias fijas, instanciación del grafo de prueba o cola concreta y declaración explícita de los valores esperados.
- **Act (Actuar):** Invocación del método o solver bajo evaluación (`dijkstra_solver`, `a_star_solver`, operaciones primitivas de cola o comandos de la CLI).
- **Assert (Comprobar):** Comparación de distancias con tolerancia numérica, invariantes de colas, excepciones esperadas y códigos de salida según el caso.

```
python -m pytest python/tests -q
```

Listing 2: Comando de ejecución de la suite automatizada completa

7.2. Tipología y Cobertura de la Suite Automatizada (349 Pruebas Python)

La suite en Python reúne 349 pruebas automatizadas en las categorías de las Tablas 12 y 13. Esta clasificación no equivale a un porcentaje de cobertura de código, que no se midió.

Tabla 12
Cobertura de las pruebas automatizadas por categoría funcional

Categoría	Módulos de Prueba	Casos y Propiedades Evaluadas
Pruebas unitarias	`test_priority_queues.py`, `test_dijkstra.py`	Invariantes de montículo, `sift-up`, `sift-down`, cortes en cascada, consolidación dinámica y contratos abstractos.
Pruebas de integración	`test_cli.py`, `test_osm_loader.py`, `test_weather.py`, `test_web_api.py`	Pipeline CLI, ingesta OSMnx, respaldo offline de clima, contratos de la API FastAPI y validación de coberturas OSM.
Validación cruzada	`test_equivalence.py`, `test_validation_io.py`	Comparación contra Bellman-Ford, Dijkstra ingenuo, NetworkX y certificado de optimalidad.
Casos límite	`test_dijkstra_manual.py`, `test_dijkstra.py`	Grafo unitario, componentes disjuntas, lazos, aristas paralelas, costos cero y pesos extremos.
Casos adversos	`test_validation_io.py`, `test_priority_queues.py`, `test_a_star.py`	Pesos negativos, ciclos negativos, archivos corruptos, claves no numéricas y heurísticas inadmisibles.

Tabla 13
Distribución modular de la suite Python de 349 pruebas

Archivo de Prueba	Cantidad	Propósito y Aserciones
`test_priority_queues.py`	61	Contrato polimórfico común, orden estricto de extracción, `decrease_key` y estrés con `heapq`.
`test_equivalence.py`	48	Equivalencia matemática de distancias con Bellman-Ford, NetworkX y Dijkstra en 40 grafos.
`test_validation_io.py`	45	Detección de JSON corruptos, entradas adversas y validación del certificado formal.
`test_dijkstra_manual.py`	36	Traza analítica de Cusco, instancia de CLRS (fig. 24.6), nodos aislados y lazos.
`test_cli.py`	25	Comandos de consola, códigos de salida (0, 1, 2) y ciclo generación–exportación.
`test_dijkstra.py`	11	Casos base de SSSP, límites y equivalencia a escala.
`test_a_star.py`	8	Poda de búsqueda, admisibilidad, consistencia y fallback cuando no hay coordenadas.
`test_weather.py`	12	Servicio climático, respuestas simuladas, manejo de 401 y modo desconectado.
`test_web_api.py`	29	Contratos FastAPI tipados, límites unificados, exportación sin recorte, geometría OSM y escenarios uniformes.
`test_traffic.py`	11	Escenarios horarios y factores de congestión vial.
`test_osm_loader.py`	4	Carga determinista de OpenStreetMap y serialización JSON.
`test_benchmark_protocol.py`	4	Estadísticas robustas (mediana e IQR) y estructura del benchmark.
`test_applied_analysis.py`	3	Consulta sobre la red real de Cusco y sensibilidad ambiental.
`test_delivery_pipeline.py`	48	Referencia exhaustiva en grafos pequeños con semillas, flujo HTTP generar–validar–resolver–verificar y rechazo de entradas no finitas en el modelo de costo.
`test_delivery_report.py`	2	Resumen JUnit que separa éxitos, fallos y omisiones; un informe vacío no se presenta como éxito.
`test_provider_keys.py`	1	Lectura de claves desde `.env` local y sobrescritura por variable de proceso, sin publicarlas.
`test_runtime_deploy.py`	1	Consistencia de archivos mínimos de despliegue (`Dockerfile`, `render.yaml` y requisitos).
Total Python	**349**	Casos recolectados y aprobados con `pytest` el 12/11/2026; comprobar en el entorno de evaluación.

7.3. Pruebas Multiplataforma (Web Vitest y Móvil Flutter)

Además de las 349 pruebas del núcleo Python, la verificación continua abarca las capas cliente en `web/tests/` y `mobile/test/` (Tabla 14). Las pruebas móviles emplean `MockClient` para simular respuestas HTTP del contrato, `sqflite_common_ffi` para probar SQLite real en memoria y pruebas de widgets con escalado tipográfico (`1.4x` y `2.0x`) para verificar diseño adaptativo sin desbordes.

Tabla 14
Pruebas automatizadas de los clientes Web (44) y Móvil Flutter (40)

Capa / Suite	Cantidad	Cobertura funcional verificada
Web (`web/tests/`, Vitest)	44	Validación de límites experimentales (5–200), generadores, componentes React, fallback estático y filtro de evidencias.
Móvil (`mobile/test/`, Flutter)	40	Contrato HTTP y manejo de errores/nulos, caché y modo sin conexión, 16 ejemplos incluidos, round-trip JSON/CSV/TXT, persistencia en SQLite interno, flujo completo de rutas/4 comparaciones, clima/experimentos, documentos offline, visualización responsive sin desbordes (`ResponsiveDataView` a 1.4x y 2.0x), validación de URLs del backend y panel institucional «Acerca de».
Total multiplataforma	**433**	Suma de Python (349) + Web (44) + Móvil (40) aprobadas al 13/11/2026 (no equivale a cobertura de líneas).

7.4. Metodología de Verificación y Oráculos Independientes

La suite emplea varias referencias en casos distintos; el banco sintético contrasta además con NetworkX antes de cronometrar. No se ejecutan todos los oráculos por cada muestra de tiempo:

1. **Algoritmo de Bellman-Ford:** Implementación pura de $O(|V||E|)$ tolerante a costos negativos y capaz de diagnosticar ciclos negativos. Se utiliza en 40 grafos aleatorios con topologías y densidades diversas.

2. **Dijkstra sobre Arreglo no Ordenado:** Algoritmo determinista $O(|V|^2 + |E|)$ que no emplea estructuras de árbol ni montículos, confirmando que la lógica de prioridades no perturba las distancias.

3. **NetworkX:** Referencia externa para distancias en pruebas y en las 30 instancias sintéticas del banco, antes de medirlas.

4. **Instancias resueltas analíticamente a mano:** Contraste contra la traza de seis nodos de Cusco (Sección 4) y la instancia presentada por Cormen et al. (2022, figura 24.6, p. 659).

7.5. Certificado de Optimalidad de Bellman

Para comprobar la salida sin repetir Dijkstra, el certificador examina distancias, aristas y predecesores; si se suministran caminos explícitos, también los recorre:

1. **Distancia al origen:** $d[s] = 0.0$.

2. **Ausencia de relajaciones pendientes:** Para toda arista dirigida $(u,v) \in E$ con $d[u] < \infty$, se comprueba, dentro de la tolerancia numérica:

$$d[v] \leq d[u] + w(u,v) \tag{26}$$

de modo que no quede una arista que reduzca el costo registrado.

3. **Predecesores:** Para cada vértice alcanzable $v \neq s$, la cadena de aristas ajustadas ($d[v] \approx d[\pi[v]] + w(\pi[v], v)$) debe terminar en s y no formar ciclos.

4. **Trayectorias, si se incluyen:** Se comprueban extremos, aristas existentes y costo, con tolerancia numérica frente a $d[v]$.

La comprobación sin caminos explícitos cuesta $O(|V| + |E|)$. Si se entregan caminos para todos los destinos, cuesta además $O(L)$, donde L es la suma de sus longitudes.

7.6. Casos Límite, Condiciones Adversas y Pruebas de Regresión

La suite de pruebas incluye escenarios extremos diseñados para someter al sistema a condiciones no nominales:

- **Casos límite:** Grafos unitarios ($|V| = 1, |E| = 0$), orígenes aislados sin aristas salientes, componentes disjuntas inalcanzables ($d[v] = \infty$), lazos dirigidos ((u, u)), aristas paralelas múltiples con distintos costos, aristas de costo cero y aristas con pesos extremos (10^{12} y 10^{-9}).
- **Casos adversos:** Entradas con pesos negativos, grafos con ciclos negativos (detectados por Bellman-Ford y rechazados por Dijkstra), JSONs con sintaxis rota, claves NaN, cadenas de texto o nulas, e intentos de incremento de clave en `decrease_key`.
- **Pruebas de A^*:** Se rechazan heurísticas personalizadas negativas, no finitas o inconsistentes; si faltan coordenadas para la heurística automática, se usa $h = 0$.

7.7. Pruebas de Integración y Funcionamiento Desconectado

La suite incluye pruebas de carga del *snapshot* OSM, clientes meteorológicos simulados y una prueba de humo de la aplicación mediante `AppTest`. Estas comprobaciones no sustituyen una instalación limpia ni una prueba de APIs externas en vivo.

7.8. Criterios de Aceptación y Calidad del Software

El control técnico requiere ejecutar `python -m pytest python/tests -q`, comprobar los certificados y la equivalencia de distancias, y probar CLI y aplicación sin red. En el banco, la verificación de la instancia precede al cronometraje; no se certifica de nuevo cada muestra temporal. La bitácora y la presentación requieren evidencias distintas de una salida de pruebas.

8. Metodología Experimental

8.1. Preguntas de Investigación e Hipótesis de Evaluación

El diseño experimental responde a tres interrogantes computacionales centrales:

1. **PI1:** ¿Qué estructura de cola de prioridad minimiza el tiempo de pared (*wall-clock time*) en el Algoritmo de Dijkstra sobre redes viales sintéticas dispersas y densas, y cómo evoluciona dicha relación frente al incremento de escala?
2. **PI2:** ¿Se traduce la cota asintótica teórica $O(1)$ amortizada en `Decrease-Key` del Fibonacci Heap en un menor tiempo real de ejecución en un entorno de software interpretado en Python 3.12?
3. **PI3:** ¿Qué medianas se observan en la consulta OSM, cuántos vértices expande A^* y cómo cambia el costo estimado con un factor climático uniforme?

8.1.1. Hipótesis de Trabajo de la Evaluación (H1 a H4)

- **H1 (Colas en redes dispersas):** En el perfil disperso, el heap binario y el 4-ario podrían registrar menores tiempos que Fibonacci. No se midieron fallos de caché ni asignaciones para atribuirles la diferencia.
- **H2 (Densidad y ramificación):** La diferencia entre 4-ario y binario puede variar con el número de aristas y la escala. El 4-ario tiene menos niveles teóricos, pero compara más hijos al extraer el mínimo.
- **H3 (Poda espacial con búsqueda heurística):** El algoritmo A^* con heurística euclidiana escalada consistente reducirá el número de expansiones de vértices respecto a Dijkstra sin alterar el costo óptimo de la ruta.
- **H4 (Factor uniforme):** Multiplicar por un mismo factor positivo todos los pesos conserva el conjunto de rutas óptimas. Solo se guardó un escenario de sensibilidad climática, no uno de congestión.

8.2. Diseño Factorial e Instancias Experimentales

El banco canónico de evaluación implementa un diseño factorial completo $3 \times 2 \times 5$:

- **Estructura de cola de prioridad (3 niveles):** Min-Heap Binario Indexado, 4-ary Heap ($d = 4$) y Fibonacci Heap.
- **Perfil de densidad del grafo (2 niveles):**
  - *Perfil disperso:* $|E| = 4|V|$, parámetro del generador sintético; no es una medición de Cusco.
  - *Perfil denso:* $|E| \approx 0.08\,|V|(|V|-1)$; el generador redondea la cantidad de aristas a un entero.
- **Escala del grafo (5 niveles por perfil):**

- Perfil disperso: $|V| \in \{100, 500, 1000, 2500, 5000\}$.
- Perfil denso: $|V| \in \{50, 100, 200, 400, 800\}$.

Por escala y perfil se generan tres instancias con semillas fijas. Cada cola se mide 15 veces por instancia, después de dos ejecuciones de calentamiento. Las mediciones repiten los mismos grafos y no son réplicas independientes de redes distintas:

$$2 \text{ perfiles} \times 5 \text{ escalas} \times 3 \text{ instancias} \times 3 \text{ colas} \times 15 \text{ réplicas} = 1,350 \text{ muestras cronometradas} \quad (27)$$

8.2.1. Generación Determinista de Instancias Sintéticas

`benchmark_network` garantiza alcance desde el origen mediante una cadena dirigida $N_0 \to N_1 \to \cdots \to N_{n-1}$; luego agrega aristas aleatorias hasta completar $|E|$. Los pesos provienen de $[1.0, 1000.0]$ y son valores sintéticos *sin unidad física*: no representan metros ni segundos de viaje. Con semilla base 12345 y $j \in \{0, 1, 2\}$ como índice de instancia:

$$s(n, j) = \begin{cases} 12345 + 31n + j, & \text{para perfil disperso,} \\ 22345 + 31n + j, & \text{para perfil denso.} \end{cases} \quad (28)$$

8.3. Variables Experimentales y Métricas de Desempeño

- **Variables independientes:** Estructura de cola, escala de vértices $|V|$, densidad de aristas $|E|$ y tipo de grafo (sintético controlado vs. OpenStreetMap real).
- **Variables dependientes:**
  - Tiempo de cómputo con reloj de pared, almacenado en microsegundos (μs) y expresado en milisegundos (ms) cuando se indica.
  - Estadísticos de tendencia central y dispersión: Mediana e Intervalo Intercuartílico (IQR), seleccionados por su robustez frente a distribuciones asimétricas y ruido del sistema operativo.
  - Conteo unívoco de primitivas algorítmicas: inserciones (`Insert`), extracciones (`Extract-Min`), decrementos efectivos de prioridad (`Decrease-Key`), intentos totales de relajación y comparaciones de clave ejecutadas.

8.4. Entorno Experimental Controlado e Instrumentación de Precisión

El banco guardado registra Python y plataforma. La Tabla 15 describe el equipo de trabajo usado; no se midió ni controló la carga del sistema o la jerarquía de caché durante cada ejecución.

Tabla 15
Especificaciones del entorno experimental de medición

Componente / Parámetro	**Especificación Técnica**
Sistema Operativo	Microsoft Windows 11 Pro 64-bit (Build 10.0.26300)
Procesador (CPU)	AMD Ryzen 5 5600H @ 3.30 GHz (Turbo hasta 4.2 GHz)
Configuración de núcleos	6 núcleos físicos, 12 subprocesos lógicos
Jerarquía de Caché	L1: 384 KB, L2: 3 MB, L3: 16 MB compartida
Memoria RAM	24 GB DDR4 @ 3200 MHz
Plataforma de Software	Python 3.12.8 (64-bit)
Frameworks Auxiliares	pytest 9.1.1, NetworkX 3.7, OSMnx 2.1.1

8.4.1. Protocolo de Aislamiento de Ruido y Precisión Temporal

1. **Temporizador:** Se usa `time.perf_counter_ns()` y se convierten los intervalos medidos a microsegundos. El nombre del reloj no implica una precisión efectiva de un nanosegundo.
2. **Recolector de basura:** Antes de cada muestra se invoca `gc.collect()` y se desactiva el recolector automático durante la llamada cronometrada; al terminar se restaura su estado anterior.
3. **Calentamiento:** Se descartan dos ejecuciones previas por combinación de instancia y cola; no se midió su efecto sobre la frecuencia del procesador ni sobre las cachés.
4. **Orden de colas:** Para cada instancia se permuta de forma reproducible el orden de las tres colas para reducir un posible sesgo de orden. Esto no elimina el ruido del sistema.
5. **Verificación previa:** Antes de medir cada instancia sintética se comparan las distancias de las tres colas, se contrastan con NetworkX y se certifica una solución. Bellman-Ford se contrasta por separado en la suite de pruebas.

8.5. Caso Aplicado sobre la Red Vial Urbana Real de Cusco

Para evaluar el comportamiento de los algoritmos en una red urbana real, se extrajo la malla vial del Centro y áreas adyacentes de la ciudad del Cusco mediante OSMnx 2.1.1 a partir de las coordenadas $(-13.5165, -71.9788)$ con un radio de 1.5 km en modalidad `drive`. La red comprende 891 intersecciones y 2,139 tramos dirigidos ($|E|/|V| = 2.40$).

El par se seleccionó con una regla fija: el origen s es el nodo más cercano al centro de extracción y el destino t es el nodo alcanzable de mayor costo temporal base. La misma consulta se midió 30 veces por cola, con tres calentamientos previos; no se muestrearon pares de rutas independientes. Para A^* se usó una heurística euclidiana escalada consistente:

$$h(u) = \lambda \cdot \|x_u - x_t\|, \quad \text{donde } \lambda = \min_{(u,v)\in E} \frac{w(u,v)}{\|x_u - x_v\|} \tag{29}$$

y se analizó la sensibilidad del trayecto bajo el factor meteorológico compuesto de Open-Meteo ($m_{\text{clima}} = 1.0691$).

8.6. Procedimiento Experimental Reproducible

El script siguiente conserva separadas la medición y la generación de tablas y figuras. Los tiempos guardados están en microsegundos; las tablas de la consulta OSM los expresan en milisegundos. El tiempo de viaje calculado para la ruta está en segundos y no mide la ejecución del programa.

```
# Regeneración sin repetir mediciones (reutiliza los 1350 datos guardados en JSON):
python python/scripts/reproduce_canonical.py --skip-benchmark --skip-applied

# Ejecución completa del benchmark canónico (1350 corridas):
python python/scripts/reproduce_canonical.py
```

Listing 3: Comando de reproducción de figuras y tablas científicas

8.7. Piloto de Verificación del Protocolo

El piloto guardado el 04/11/2026 verificó el protocolo en dos grafos de 100 vértices: uno disperso (400 aristas) y uno denso (792). Se usó una instancia por perfil, tres colas y tres mediciones por cola, con dos calentamientos previos: 18 muestras cronometradas. El script comprobó las distancias frente a NetworkX y el certificado antes de medir. `python/data/processed/pilot_benchmark_results.json` conserva muestras, semillas y estadísticos; la Figura 5 muestra esas muestras. El piloto se hizo *después* del banco canónico fechado el 28/10/2026: comprueba que el procedimiento se puede repetir, pero sus tres mediciones por cola no bastan para decidir qué cola rinde mejor. La semilla reproduce los grafos, no el tiempo de reloj. Desde la raíz del repositorio:

```
python python/scripts/reproduce_pilot.py
# Solo regenerar la figura sin repetir mediciones:
python python/scripts/reproduce_pilot.py --plot-only
```

Listing 4: Reproducción del ensayo piloto y su figura

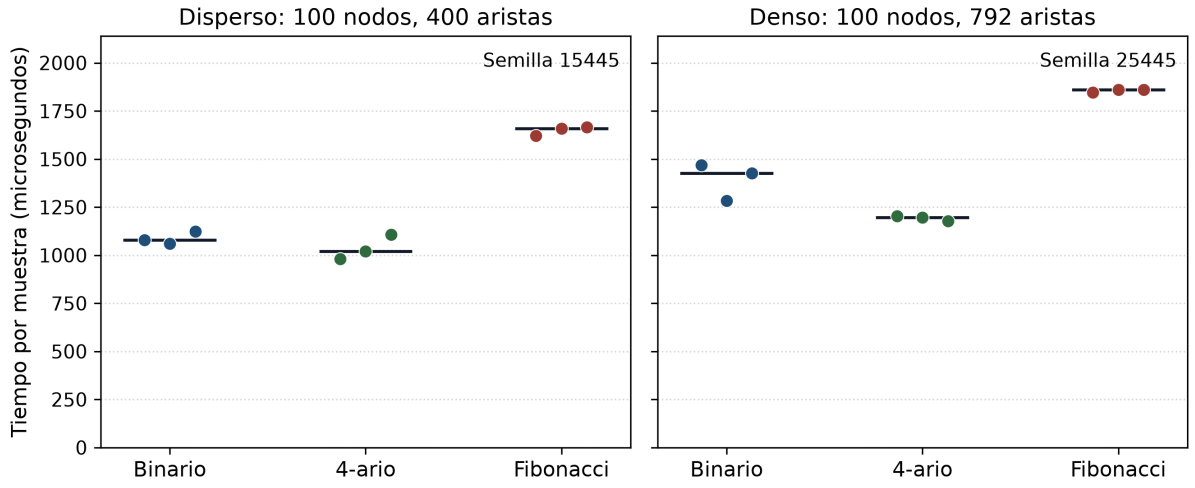


Figura 5
Piloto del 04/11/2026. Cada punto representa una medición; el segmento, la mediana de tres repeticiones. Datos: `pilot_benchmark_results.json`*. La comparación por escenario se presenta en la Sección 9.*

8.8. Amenazas a la Validez

- **Validez interna:** El planificador y la frecuencia del procesador pueden alterar los tiempos. El calentamiento y la desactivación del GC reducen algunas fuentes de variación, pero no las eliminan. Mediana e IQR describen las muestras; no corrigen el ruido.
- **Validez externa:** Las mediciones proceden de Python 3.12 en un procesador AMD Ryzen. No se puede extrapolar la diferencia observada a otros lenguajes o equipos sin repetir el experimento.
- **Validez de conclusión:** El solapamiento de IQR describe dispersión, pero por sí solo no es una prueba de significación estadística ni de igualdad entre colas. No se midieron contadores físicos de caché ni consumo de memoria.

9. Resultados

9.1. Verificación de Corrección y Equivalencia de Distancias

Antes de medir, se resolvieron las 30 instancias sintéticas con las tres colas y se comprobaron sus distancias entre sí y frente a NetworkX. Una de las soluciones por instancia pasó el certificado de optimalidad. En la consulta OSM se verificaron la equivalencia y los certificados de las tres colas. Las 1,350 muestras del banco corresponden a tiempos de ejecución, no a 1,350 verificaciones independientes; Bellman-Ford se usa como oráculo en la suite de pruebas (Sección 7).

9.2. Benchmark Canónico en Instancias Sintéticas (Perfiles Disperso y Denso)

Las Tablas 16 y 17 presentan medianas e intervalos intercuartílicos (IQR) de 45 muestras por escala y cola (3 instancias generadas $\times$ 15 mediciones por instancia). La Figura 6 muestra ambos perfiles en escala logarítmica.

Tabla 16
Dijkstra con tres colas, perfil disperso: mediana e IQR en μs.

$\|V\|$	$\|E\|$	Binary Heap	4-ary Heap	Fibonacci Heap	Mejor
100	400	1648.4 [1377.9, 1979.3]	1673.3 [1393.8, 2186.5]	2574.3 [2371.2, 2870.8]	Binary
500	2000	19232.5 [14655.3, 24254.5]	14531.3 [11524.1, 18064.2]	30497.4 [20429.6, 38086.3]	4-ary
1000	4000	43378.1 [37780.5, 54962.6]	41803.4 [33976.3, 53954.7]	58348.7 [47548.7, 65855.4]	4-ary
2500	10000	105786.6 [96970.5, 121834.1]	106339.6 [94379.9, 122938.1]	138649.1 [121508.9, 170908.3]	Binary
5000	20000	254749.2 [219442.6, 316895.6]	212803.0 [199918.1, 232243.5]	285718.4 [261162.8, 315459.6]	4-ary

Nota. Cada celda muestra la mediana y, entre corchetes, el IQR.

Tabla 17
Dijkstra con tres colas, perfil denso: mediana e IQR en μs.

$\|V\|$	$\|E\|$	Binary Heap	4-ary Heap	Fibonacci Heap	Mejor
50	196	892.0 [821.0, 1074.6]	872.9 [783.2, 994.0]	1284.4 [1132.8, 1411.3]	4-ary
100	792	2287.0 [2057.4, 3227.4]	2182.2 [1894.3, 2405.6]	3142.8 [2655.2, 3976.6]	4-ary
200	3184	5138.6 [4040.9, 6430.3]	4852.3 [3881.0, 6065.4]	7169.3 [5476.0, 8327.4]	4-ary
400	12768	16408.9 [13090.2, 18599.3]	15487.6 [12208.9, 19884.7]	17183.7 [15487.0, 21265.9]	4-ary
800	51136	57923.5 [50208.7, 62974.9]	58345.2 [50691.3, 68398.6]	74614.7 [60800.1, 89075.6]	Binary

Nota. Cada celda muestra la mediana y, entre corchetes, el IQR.

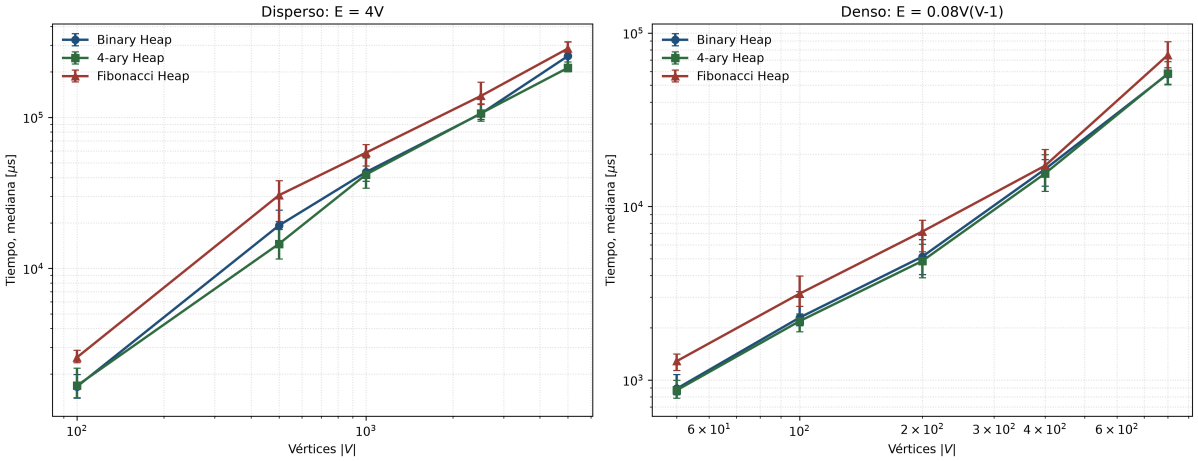


Figura 6
Tiempo mediano de ejecución de Dijkstra con las tres colas de prioridad evaluadas. Izquierda: perfil disperso ($|E| = 4|V|$). Derecha: perfil denso ($|E| = 0.08\,|V|(|V|-1)$). Las barras de error representan el Intervalo Intercuartílico (IQR).

9.2.1. Análisis del Perfil Disperso ($|E| = 4|V|$)

El montículo 4-ario (`DAryHeap`) obtuvo la menor mediana en 3 de las 5 escalas ($|V| \in \{500, 1000, 5000\}$): 24.4 %, 3.6 % y 16.5 % menos tiempo que el binario, respectivamente. El binario registró la menor mediana con $|V| = 100$ y $|V| = 2500$. La mediana de las cinco razones 4-ario/binario fue 0.964; no es una razón de tiempos agregados. Los IQR se solapan en las cinco escalas, por lo que estas diferencias se describen como observaciones, no como una ventaja demostrada para cualquier instancia dispersa.

9.2.2. Análisis del Perfil Denso ($|E| = 0.08\,|V|(|V|-1)$)

En el perfil denso, el 4-ario registró la menor mediana en cuatro escalas ($|V| \in \{50, 100, 200, 400\}$); el binario lo hizo en $|V| = 800$. La mediana de las cinco razones 4-ario/binario fue 0.954. También aquí los IQR se solapan en todas las escalas.

9.2.3. Desempeño Comparativo del Fibonacci Heap

El `FibonacciHeap` tuvo la mayor mediana en las diez escalas: entre 1.12 y 1.59 veces la del binario en el perfil disperso, y entre 1.05 y 1.44 en el denso. En ocho escalas sus IQR se solapan con los del binario. Sus operaciones sobre nodos enlazados podrían contribuir al costo observado; no se midieron asignaciones ni fallos de caché para separar esas causas.

Tabla 18
Síntesis por perfil y consulta: conteo de menores medianas observadas, sin declarar diferencias concluyentes donde se solapan los IQR.

Perfil	**4-ary/Binary**	**Fibonacci/Binary**	Menor mediana 4-ary	Menor mediana Binary	**IQR solap.**
Disperso ($\|E\| = 4\|V\|$)	0.964	1.345	3 de 5	2 de 5	5 de 5
Denso ($\|E\| = 0.08\|V\|(\|V\|-1)$)	0.954	1.374	4 de 5	1 de 5	5 de 5
Total sintético	0.959	1.360	7 de 10	3 de 10	10 de 10
Consulta OSM	1.269	1.666	0	1	1 de 1

Nota. Razón 4-ary/Binary menor que 1: menor mediana del 4-ario. IQR solap. compara 4-ary frente a Binary; el solapamiento impide afirmar superioridad inequívoca. Fibonacci no obtuvo la menor mediana en ninguna escala sintética.

Respuesta a la comparación. En el banco sintético, el heap 4-ario obtuvo la menor mediana en 7 de 10 escalas; en la consulta OSM de Cusco, el binario registró la menor mediana (15.65 ms frente a 19.87 ms del 4-ario y 26.09 ms del Fibonacci). Fibonacci no obtuvo la menor mediana en ninguna escala. Estas son diferencias *observadas por escenario*, no una victoria universal: los IQR del binario y el 4-ario se solapan en las diez escalas sintéticas y en la consulta OSM (Tablas 18 y 19). El piloto de la Sección 8 verifica el procedimiento; la evaluación comparativa utiliza las 1,350 muestras del banco completo y las 30 réplicas del caso OSM.

9.3. Análisis Cuantitativo de Operaciones Elementales y Comparaciones

La Figura 7 expone el desglose de primitivas computacionales ejecutadas en el perfil disperso. En la mayor escala ($|V| = 5000, |E| = 20000$), las tres estructuras registraron exactamente 5,000 inserciones, 5,000 extracciones de mínimo y 6,695 decrementos efectivos de prioridad sobre un total de 10,415 intentos de relajación.

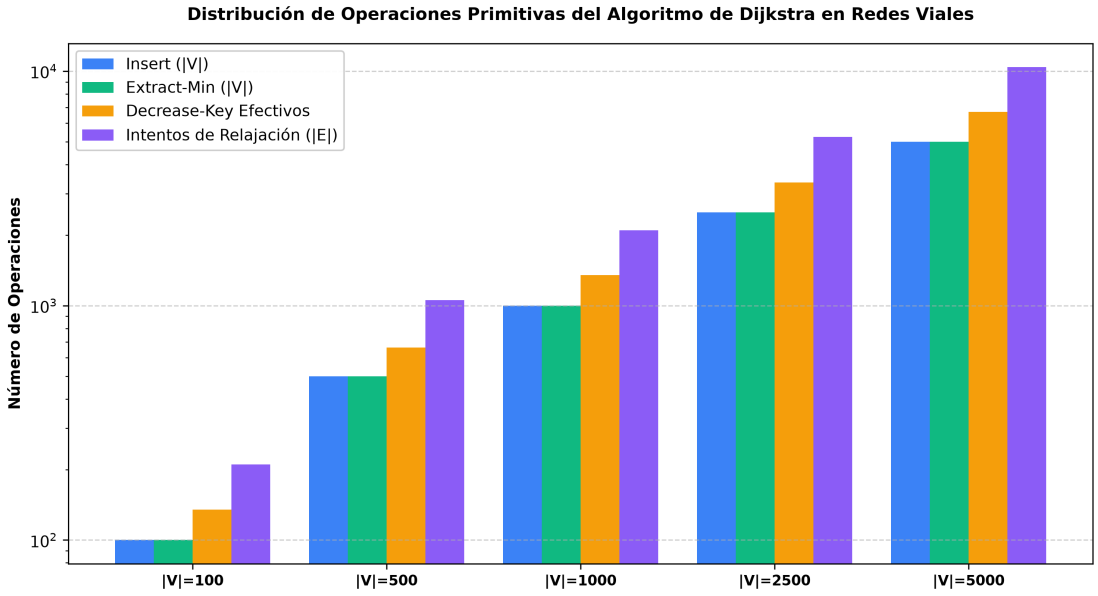


Figura 7
Distribución de operaciones elementales de Dijkstra en el perfil disperso sintético. Se discriminan inserciones, extracciones, decrementos de clave efectivos e intentos totales de relajación.

En $|V| = 5000$ disperso, el Fibonacci Heap ejecutó 101,303 comparaciones frente a 125,776 del binario (19.5 % menos), pero su mediana de tiempo fue 12.2 % mayor. Con $|V| = 800$ denso, registró 19,848 frente a 21,280 comparaciones (6.7 % menos) y una mediana 28.8 % mayor. Menos comparaciones no implicaron menos tiempo en estas instancias; los contadores no separan el costo de asignaciones, cortes y accesos a memoria.

9.4. Caso de Estudio Aplicado: Red Vial Real de Cusco (OpenStreetMap)

El recorte vial OSM de Cusco contiene 891 nodos y 2,139 aristas dirigidas. El origen es el nodo más próximo al centro elegido para la descarga (`30972889`); el destino, el nodo alcanzable de mayor tiempo nominal desde él (`5996728243`). La ruta obtenida contiene 17 vértices, suma 2,810.6 metros y tiene un costo estimado de 387.197 segundos. La Figura 8 muestra su trazado sobre el grafo guardado.

Como referencia geográfica del área consultada, usamos la vista de OpenStreetMap disponible en este enlace. La captura muestra las calles y lugares del Centro Histórico que sirven para ubicar visualmente la zona; no constituye la salida del algoritmo ni reemplaza el *snapshot* utilizado en las mediciones.

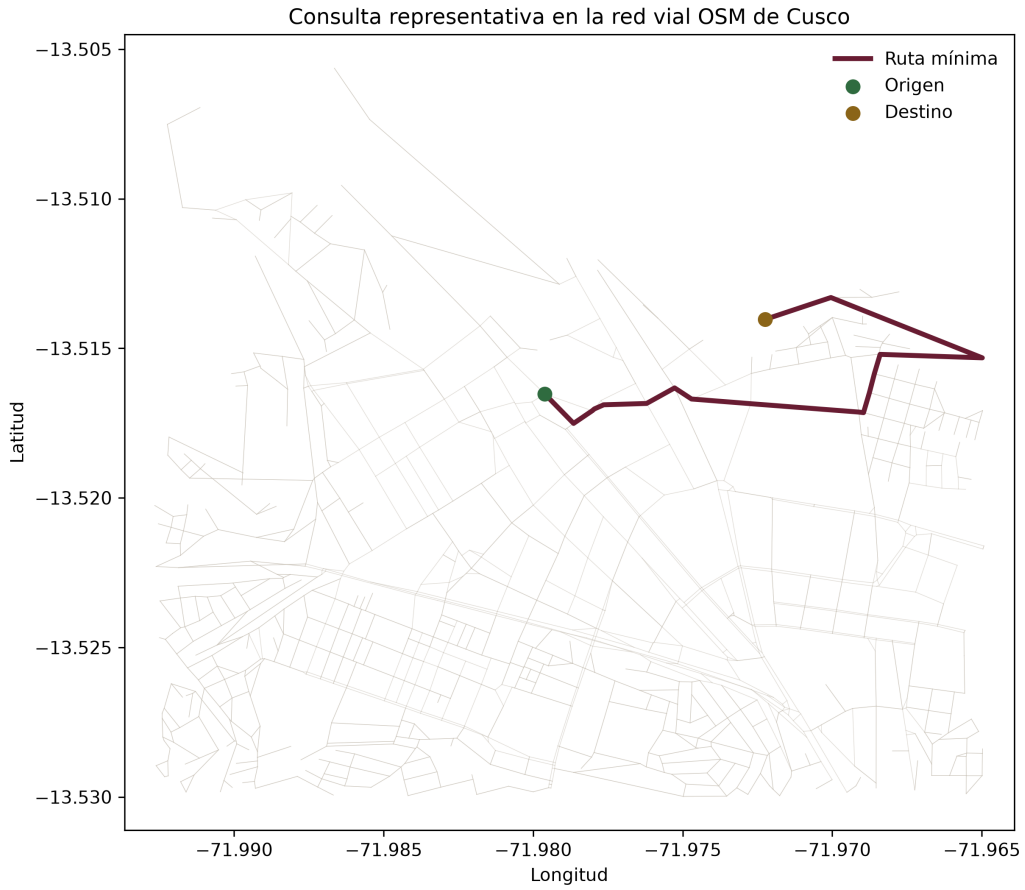


Figura 8

Ruta de menor tiempo nominal sobre el recorte OpenStreetMap de Cusco. La línea representa el camino calculado y los marcadores señalan origen y destino.

La captura de referencia se incorpora en la Figura 9. La red que procesamos no se leyó manualmente de la imagen: la descargamos mediante OSMnx desde OpenStreetMap alrededor de `(-13.5165, -71.9788)`, con un radio de 1,500 metros y el tipo de red `drive`. Después congelamos el resultado en el archivo `cusco_osm_drive.json` para que las mediciones fueran reproducibles. Este archivo contiene los nodos, las coordenadas y las aristas dirigidas con sus longitudes y tiempos nominales; no contiene pendientes medidas ni tráfico real por tramo.

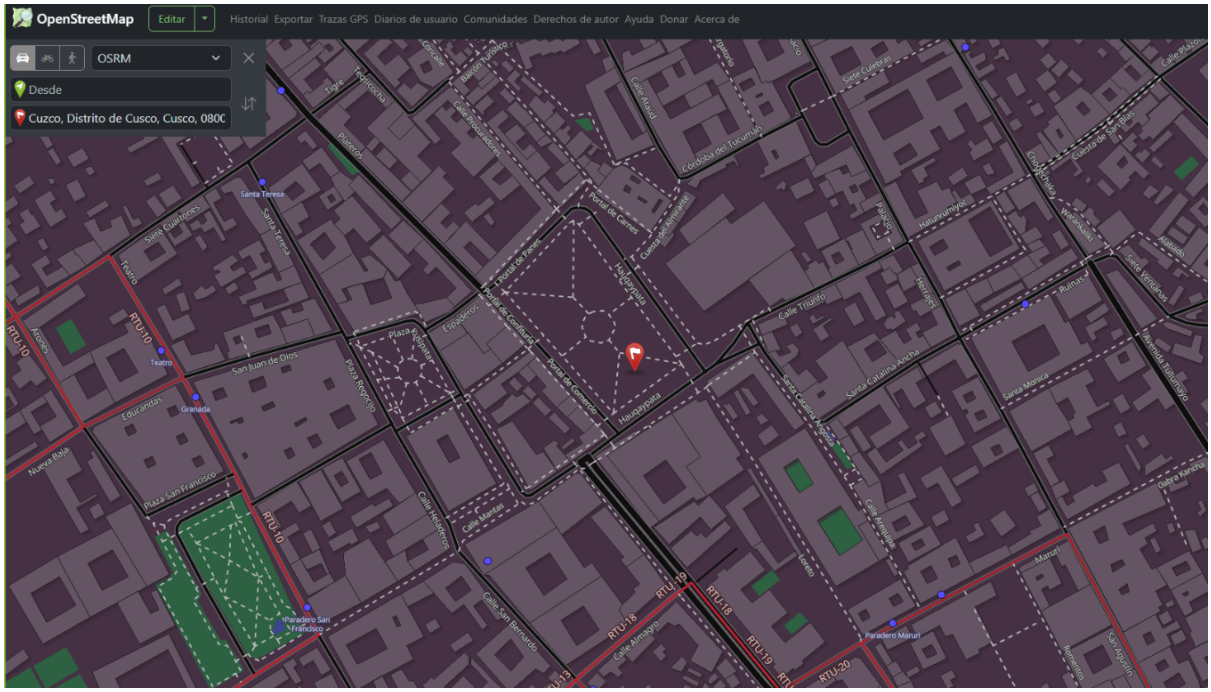


Figura 9
Vista de referencia del Centro Histórico del Cusco en OpenStreetMap. La imagen se utiliza para identificar visualmente el área de estudio y no representa la ruta calculada por el programa.

La Tabla 19 resume los tiempos de cómputo para esa consulta, no el tiempo de viaje de 387.197 segundos. El binario registró la menor mediana: 15.65 ms, frente a 19.87 ms del 4-ario (+26.9 %) y 26.09 ms del Fibonacci (+66.7 %). Los IQR del binario y el 4-ario se solapan; la posible ventaja por menor sobrecarga de índices no se midió por separado.

Tabla 19
Caso OSM: tiempo de Dijkstra por cola, mediana e IQR.

Cola	Mediana (μs)	Q1 (μs)	Q3 (μs)
Binary Heap	15655.0	13065.6	19550.7
4-ary Heap	19872.2	17335.0	22675.6
Fibonacci Heap	26088.7	24033.9	27343.9

Tabla 20
Sensibilidad meteorológica uniforme en la consulta OSM.

Escenario	Tiempo (s)	Cambio (%)
Base	387.20	0.00
Snapshot meteorológico	413.95	6.91

9.5. Extensión Algorítmica con Búsqueda Heurística A^*

Con la heurística euclidiana escalada ($\lambda = 6484.687$), A^* devolvió el mismo costo para el par consultado que Dijkstra: 387.197 segundos. Expandió 823 vértices, frente a 887 de Dijkstra (64 menos, o 7.2 %). El JSON guarda la igualdad de costos y los contadores, no la ruta obtenida por A^* ni su tiempo de cómputo dentro del banco de colas.

9.6. Ampliación del Caso de Estudio: Escala Urbana (1.5 km vs. 3.0 km) e Integración de Condiciones

Para evaluar la respuesta algorítmica ante una expansión del dominio espacial sin alterar el marco canónico, el sistema incorpora dos recortes viales estructurados mediante OSMnx:

1. **Malla Central (1.5 km):** Centrada en $(-13.5165, -71.9788)$, cubre el Centro Histórico y vías colectoras con 891 intersecciones y 2 148 aristas dirigidas ($|E|/|V| = 2.41$).
2. **Red Urbana Ampliada (3.0 km):** Amplía el radio a 3 000 m con 1 938 nodos y 4 798 aristas ($|E|/|V| = 2.48$), integrando avenidas periféricas y mayor redundancia de desvíos.

La Tabla 21 sintetiza la comparación estructural y de desempeño entre ambas escalas para la misma consulta canónica ($s =$ `30972889` $\rightarrow t =$ `5996728243`). En la red de 3.0 km, el tiempo de cálculo de Dijkstra pasa de 11.84 ms a 30.40 ms (factor $\times 2.57$), mientras que el costo óptimo de viaje resultante es de 389.69 s a través de 18 vértices (frente a 387.20 s y 17 vértices en la red central), reflejando la convergencia sobre arterias perimétricas de mayor fluidez.

Tabla 21
Comparación entre el recorte central (1.5 km) y la red urbana ampliada (3.0 km) de Cusco

Atributo / Parámetro	Centro Cusco (1.5 km)	Red Ampliada (3.0 km)	Comportamiento en Interfaces
Archivo en repositorio	`cusco_osm_drive.json`	`cusco_urbano_drive.json`	Conmutación instantánea mediante selectores en cabecera.
Vértices ($\|V\|$)	891 intersecciones	1 938 intersecciones	Renderizado interactivo sobre mapa vectorial.
Aristas dirigidas ($\|E\|$)	2 148 tramos	4 798 tramos	Validación sub-segundo con cache local.
Densidad ($\|E\|/\|V\|$)	2.41 aristas/nodo	2.48 aristas/nodo	Topología cuasi-plana vial típica.
Costo óptimo base ($s \rightarrow t$)	387.20 s (17 nodos, 2 811 m)	389.69 s (18 nodos, 2 844 m)	Equivalencia certificada entre Dijkstra y A^*.
Tiempo cómputo solver	11.84 ms	30.40 ms	Medido en servidor Python (FastAPI).
Condiciones dinámicas	Clima + Tráfico horario	Clima + Tráfico horario	Ajustes aplicables en vivo o por histórico.

Ejemplo numérico de cálculo de peso con impedancia dinámica. Para ilustrar la aplicación de la función de costo formulada en la Ecuación (17), considérese un tramo vial representativo en el Centro Histórico (e.g., Cuesta San Blas, arista $u \rightarrow v$):

- Parámetros físicos base: longitud $L = 120$ m, tipología `calle_centro` con velocidad nominal $v_{\text{nom}} = 15$ km/h (4.167 m/s), y pendiente ascendente $\theta = +8\,\%$ (0.08).

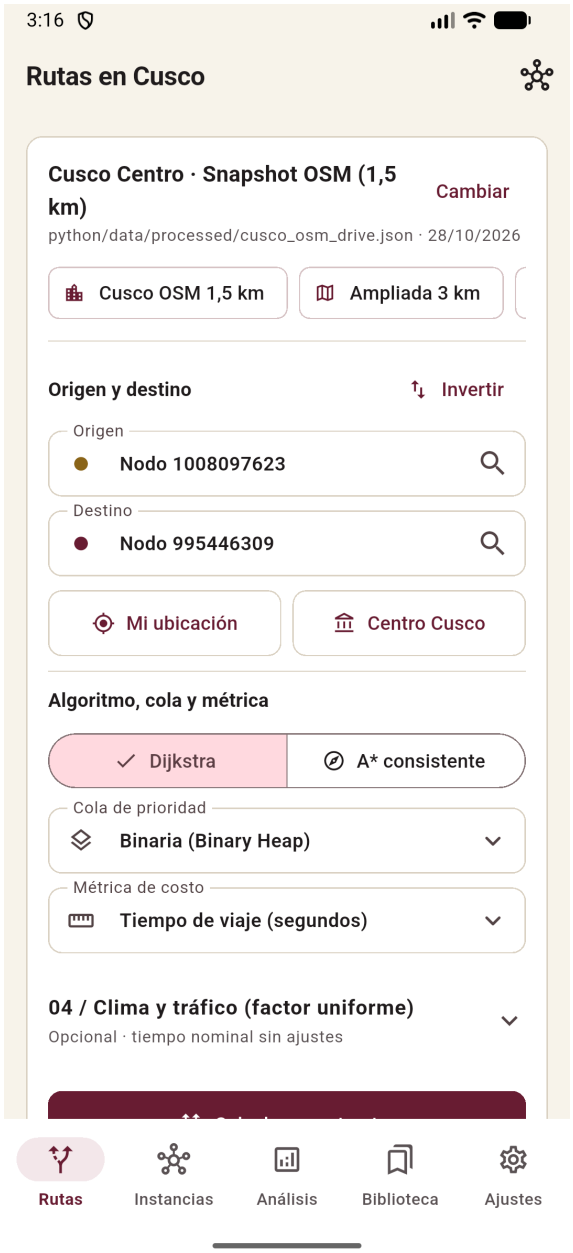
- Tiempo de viaje nominal plano: $T_{\text{base}} = \frac{120\text{ m}}{4.167\text{ m/s}} = 28.80$ s.
- Penalización topográfica ($\alpha = 2.5$): $f_{\text{topo}} = 1 + 2.5 \times 0.08 = 1.20 \implies T_{\text{topo}} = 28.80 \times 1.20 = 34.56$ s ($+20.0\,\%$).
- Multiplicador climático ($P = 0.1$ mm, viento 40.7 km/h, visibilidad 8 920 m): $m_{\text{clima}} = 1.0691 \implies T_{\text{clima}} = 34.56 \times 1.0691 = 36.95$ s ($+6.91\,\%$).
- Coeficiente de congestión horaria (hora punta matutina 08:30, $c_{\text{tráfico}} = 1.25$):

$$w(u,v) = T_{\text{base}} \cdot f_{\text{topo}} \cdot m_{\text{clima}} \cdot c_{\text{tráfico}} = 28.80 \times 1.20 \times 1.0691 \times 1.25 = 46.19\text{ s}. \tag{30}$$

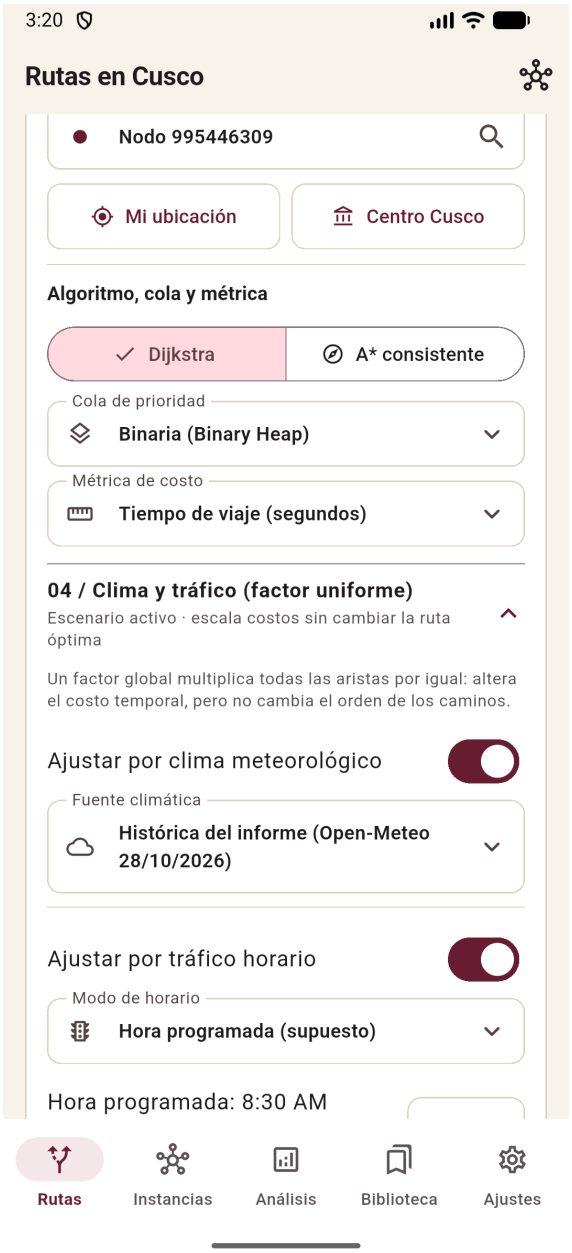
  El costo del tramo se incrementa un $+60.38\,\%$ respecto al valor plano (28.80 s $\rightarrow$ 46.19 s), reflejando el impacto acumulado de gradiente, fricción atmosférica y hora pico.

Integración operativa en las interfaces Web y Móvil. Tanto en la plataforma Web React como en la aplicación móvil Flutter (`mobile/`), el usuario dispone de controles visuales integrados (Figura 10):

- **Conmutación de escala de red (Figura 10a):** La cabecera del módulo de Rutas ofrece selectores dedicados (`[Cusco OSM 1,5 km]` y `[Ampliada 3 km]`), permitiendo alternar entre el casco monumental y la red urbana ampliada sin recarga ni pérdida del origen/destino seleccionado.
- **Panel interactivo de condiciones (Figura 10b):** El acordeón desplegable permite activar independientemente el ajuste climático (conmutando entre la observación histórica congelada del 28/10/2026 y mediciones en vivo vía Open-Meteo o OpenWeatherMap) y el tráfico vehicular (simulación horaria según franja horaria programada o en vivo mediante TomTom).



(a) *Selección de escala (1.5 km vs 3.0 km) y parámetros.*



(b) *Panel interactivo de clima histórico y tráfico horario.*

Figura 10

Integración de modos de escala e interfaces de condiciones ambientales en la aplicación móvil Flutter. A la izquierda, alternancia rápida entre el recorte de 1.5 km y la red ampliada de 3.0 km con controles de búsqueda y visualización. A la derecha, configuración modular del escenario climático (Open-Meteo) y congestión horaria programada.

10. Discusión Crítica

10.1. Contraste entre Teoría Asintótica y Desempeño Empírico

La cota amortizada de Dijkstra con Fibonacci Heap mejora la dependencia respecto de $|E|$ frente a un heap binario:

$$T_{\text{Fib}} = O(|E| + |V| \log |V|) \quad \text{frente a} \quad T_{\text{Bin}} = O((|V| + |E|) \log |V|) \tag{31}$$

Esa diferencia asintótica importa cuando crece el número de aristas, pero no fija las constantes de una implementación. En las diez escalas sintéticas medidas en Python 3.12, el Fibonacci Heap registró medianas entre 1.05 y 1.59 veces las del binario.

El Fibonacci Heap manipula nodos enlazados y consolida árboles al extraer el mínimo. Es una posible fuente de sobrecosto, pero no se perfilaron asignaciones ni memoria; los tiempos y contadores de operaciones no permiten cuantificarla.

10.2. Montículo 4-ario: cotas y mediciones

El 4-ario obtuvo la menor mediana en 7 de 10 escalas sintéticas. La mediana de las razones 4-ario/binario fue 0.964 en el perfil disperso y 0.954 en el denso. Su menor profundidad teórica puede reducir el recorrido de `decrease_key`; `extract_min`, en cambio, compara hasta cuatro hijos por nivel. Los elementos están contiguos en el arreglo, pero no medimos fallos de caché. Además, los IQR de ambos montículos se solapan en las diez escalas: estos datos no bastan para atribuir la diferencia a la caché ni para afirmar una ventaja general del 4-ario.

10.3. Desempeño del Montículo Binario en la Red Real de Cusco

En la consulta OSM de Cusco (891 intersecciones y 2,139 aristas), el binario registró la menor mediana de cómputo: 15.65 ms, frente a 19.87 ms del 4-ario y 26.09 ms del Fibonacci. Se trata de un par origen–destino en una sola red; no prueba que el binario sea siempre más rápido por debajo de mil vértices. En particular, el 4-ario tuvo menor mediana en algunas escalas sintéticas menores que mil.

10.4. Influencia de la Densidad del Grafo y Operaciones Decrease-Key

En el recorte OSM se registran 2.40 aristas dirigidas por vértice. Para $|V| = 5000$ disperso, el banco contabilizó 6,695 decrementos de clave entre 10,415 intentos de relajación (64.3 %). Este recuento depende de la instancia; por sí solo no explica la diferencia de tiempos entre colas.

10.5. Evaluación de la Poda Espacial con Búsqueda Heurística A^*

En el mismo par OSM, A^* expandió 823 vértices frente a 887 de Dijkstra (7.2 % menos) y devolvió el mismo costo de viaje, 387.197 s. No se midió el tiempo de cómputo de A^* como parte de la comparación de colas; reducir expansiones no demuestra por sí solo una reducción de tiempo. Tampoco se evaluaron pendientes ni pasajes peatonales en el *snapshot* vial.

10.6. Análisis de Sensibilidad de Factores Ambientales e Invariancia de Ruta

La simulación con el factor uniforme $m_{\text{clima}} = 1.0691$ conserva el orden de costos entre caminos:

$$w_{\text{clima}}(e) = m_{\text{clima}} \cdot w(e), \quad \forall e \in E \implies \sum_{e \in P_1} w_{\text{clima}}(e) < \sum_{e \in P_2} w_{\text{clima}}(e) \iff \sum_{e \in P_1} w(e) < \sum_{e \in P_2} w(e) \quad (32)$$

El costo calculado pasó de 387.197 s a 413.952 s y la ruta de 17 vértices permaneció igual. Es un escenario uniforme basado en una observación meteorológica guardada, no una medición de tráfico ni una estimación calibrada por tramo.

10.7. Guía Práctica y Marco de Decisión para Selección de Colas

Para esta implementación en Python, las mediciones sugieren lo siguiente:

1. **Binario:** Punto de partida sencillo; tuvo la menor mediana en la consulta OSM y en tres escalas sintéticas.
2. **4-ario:** Conviene incluirlo al repetir mediciones en otras redes; registró menor mediana en siete escalas sintéticas, con IQR solapados frente al binario.
3. **Fibonacci:** Conserva el interés por su cota amortizada, aunque no ganó ninguna de las escalas medidas aquí. No extrapolamos este resultado a C++ ni a otros tipos de grafo.
4. **A^*:** Permite explorar menos vértices en la consulta OSM evaluada sin cambiar el costo; sus tiempos requieren una medición aparte.

10.8. Limitaciones del Estudio y Amenazas a la Validez

- **Intérprete y lenguaje:** Los tiempos se midieron en Python 3.12.8 sobre un AMD Ryzen 5 5600H. Otros equipos e intérpretes pueden dar resultados distintos.
- **Ausencia de mediciones físicas de memoria:** Las comparaciones espaciales se fundamentan en las cotas teóricas de punteros y estructuras de datos, al no haberse instrumentado un perfilador residente de asignación a nivel de sistema operativo.
- **Escenarios ambientales uniformes:** Los multiplicadores climáticos y de congestión operan como factores de sensibilidad homogéneos; futuros estudios requerirán calibrar coeficientes empíricos diferenciados por segmento vial a partir de aforos vehiculares en campo.

11. Conclusiones y Recomendaciones

Balance de Objetivos Específicos (OE1–OE4)

El trabajo permite resumir los cuatro objetivos así:

1. **OE1 (Modelar):** Se implementaron dos ámbitos distintos: una instancia didáctica con pendientes supuestas y un *snapshot* OSM sin pendientes medidas ni congestión por arista. El escenario climático escala de manera uniforme los tiempos base.
2. **OE2 (Implementar):** Dijkstra utiliza tres colas intercambiables en Python; A^* se evalúa aparte para la consulta punto a punto.
3. **OE3 (Verificar):** La suite documentada reúne 349 pruebas Python (complementadas por 44 pruebas web y 40 pruebas móviles Flutter). El banco compara las tres colas y NetworkX antes de medir, y certifica una solución por instancia; Bellman-Ford se usa en las pruebas independientes. La ejecución de la suite debe comprobarse en el entorno en que se presente el sistema.
4. **OE4 (Evaluar):** El banco conservado tiene 1,350 muestras temporales y diez escalas sintéticas. La consulta OSM añade 30 mediciones por cola para un par origen–destino. Los resultados no permiten extrapolar el orden de rendimiento a otras máquinas o redes.

Respuesta a las Preguntas de Investigación

PI1. ¿Qué cola minimiza el tiempo de pared y cómo evoluciona con la escala?

El 4-ario registró la menor mediana en 7 de 10 escalas sintéticas; el binario, en 3 de 10 y en la consulta OSM (15.65 ms). Los IQR de ambos se solapan en todas las escalas y en OSM. Ni el tamaño por sí solo ni la cota asintótica permiten fijar un ganador general. No se midieron fallos de caché.

PI2. ¿Se traduce la ventaja asintótica del Fibonacci Heap en tiempo de pared?

No en las escalas evaluadas con esta implementación Python. La mediana del Fibonacci Heap fue entre 1.05 y 1.59 veces la del binario. Con 5,000 vértices dispersos contó 19.5 % menos comparaciones de claves, pero eso no bastó para reducir el tiempo de cómputo. El origen exacto de la diferencia requiere perfilado adicional.

PI3. ¿Qué aportan la extensión A^* y el análisis de sensibilidad climática?

En el par OSM evaluado, A^* expandió 7.2 % menos vértices y conservó el costo de ruta de Dijkstra. El factor climático uniforme aumentó el tiempo de viaje calculado un 6.91 % sin cambiar la ruta; no representa congestión observada por tramo.

Conclusión Principal y Síntesis de Contribuciones

Los tiempos observados dependen de la implementación, la instancia y el equipo de medición. Para volver a contrastarlos se conservan:

- Un núcleo algorítmico desacoplado, modular y reproducible en Python 3.12 con tres colas intercambiables y extensión A^*.

- Un certificado de optimalidad que comprueba aristas y predecesores; su costo es $O(|V|+|E|+L)$ si también se verifican caminos explícitos de longitud total L.
- Una suite de 349 pruebas unitarias y de integración en `python/tests/`, junto con 44 pruebas web en `web/tests/` y 40 pruebas móviles en `mobile/test/` (433 pruebas automatizadas en total).
- Una aplicación web React/TypeScript desplegada en GitHub Pages conectada a la API FastAPI (`v1.5.0`) en Render con visores MapLibre.
- Una aplicación móvil Flutter/Dart (`mobile/`) para Android (y estructura compatible con iOS) que consume el mismo contrato FastAPI, integra ejemplos, editor táctil, mapa/grafo, experimentos, documentación sin conexión y almacenamiento interno SQLite en el dispositivo.

Recomendaciones para la Práctica de Ingeniería

1. **Colas:** Empezar por el binario y medir también el 4-ario sobre las instancias concretas de uso. No descartar otras implementaciones de Fibonacci sin evaluarlas.
2. **Navegación punto a punto:** Comprobar la consistencia de la heurística antes de usar A^* y medir su tiempo si se quiere comparar rendimiento.
3. **Medición:** Conservar muestras, semillas y entorno; usar mediana e IQR sin tratarlos como pruebas de significación estadística.

Líneas de Trabajo Futuro y Posibles Extensiones

1. Repetir el banco con implementaciones en otros lenguajes y equipos para comprobar si se mantiene el orden de tiempos observado en Python.
2. Perfilar asignaciones y accesos a memoria, y medir fallos de caché con herramientas apropiadas antes de atribuirles una diferencia de rendimiento.
3. Calibrar coeficientes de impedancia meteorológica y congestión diferenciados por segmento vial a partir de aforos de tránsito en tiempo real en la ciudad del Cusco.
4. Completar la compilación, firma y distribución de la variante iOS de `mobile/` en un entorno macOS con Xcode, además de pruebas de campo adicionales sobre teléfonos Android físicos mediante ADB.

Lecciones Aprendidas

El análisis separa corrección, tiempo de cómputo y costo estimado de viaje. Los contadores de operaciones ayudan a interpretar los resultados, pero no reemplazan mediciones de memoria o caché. El aprendizaje de cada integrante requiere registros propios.

12. Referencias

Boeing, G. (2025). Modeling and Analyzing Urban Networks and Amenities with OSMnx. *Geographical Analysis*, *57*(4), 567-577. https://doi.org/10.1111/gean.70009

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*(1), 269-271. https://doi.org/10.1007/BF01386390

Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, *34*(3), 596-615. https://doi.org/10.1145/28869.28874

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, *4*(2), 100-107. https://doi.org/10.1109/TSSC.1968.300136

Open-Meteo. (2026). *Open-Meteo Weather Forecast API Documentation*. Consultado el 28 de octubre de 2026, desde https://open-meteo.com/

OpenStreetMap contributors. (2026). *Planet dump retrieved from OpenStreetMap*. Consultado el 28 de octubre de 2026, desde https://www.openstreetmap.org

OpenWeather. (2026). *OpenWeatherMap Current Weather Data API Documentation*. Consultado el 31 de octubre de 2026, desde https://openweathermap.org/current

Tarjan, R. E. (1983). *Data Structures and Network Algorithms*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9781611970265

13. Anexos

Anexo A: Instalación y configuración

Los resultados guardados se obtuvieron con Python 3.12.8 en Windows 11. Desde la raíz del repositorio se puede preparar el entorno con estos comandos; la instalación en un equipo limpio debe comprobarse por separado:

```
git clone https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC.git
cd PROYECTO_Algoritmos-Avanzados-UNSAAC
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r python/requirements.txt
```

Listing 5: Clonación, entorno virtual e instalación

Los requisitos principales son Python 3.10 o superior, `pytest`, `networkx`, `osmnx`, `fastapi`, `uvicorn`, `pandas`, `numpy` y `matplotlib`; la interfaz web requiere Node.js 22+ y las dependencias de `web/`; la aplicación móvil en `mobile/` requiere Flutter 3.47+ (Dart 3.13+) y Android SDK (o macOS con Xcode para compilar iOS). La compilación del informe requiere una distribución LaTeX con `pdflatex` y `biber`. La red OSM y el clima se versionan como *snapshots*; no es obligatorio descargarlos para ejecutar las pruebas.

Anexo B: Uso del prototipo

Suites de pruebas. Desde la raíz o el módulo correspondiente:

```
.\.venv\Scripts\python.exe -m pytest python/tests -q
cd web && npm test && npm run build
cd mobile && flutter analyze && flutter test
```

Listing 6: Ejecución de las suites automatizadas (349 Python, 44 web y 40 móvil)

Configuración de credenciales de API (opcional). Tras revocar cualquier clave expuesta, configure una clave nueva y propia. Para la CLI se admite la variable de entorno, nunca un archivo versionado (https://home.openweathermap.org/api_keys):

```
# PowerShell en Windows
$env:OPENWEATHERMAP_API_KEY = "SU_CLAVE_NUEVA"
```

Listing 7: Definición de variable de entorno para OpenWeatherMap

La web no toma automáticamente la variable ni archivos locales: exige introducir una clave nueva en el campo de contraseña de la consulta opcional. Si el proveedor rechaza la conexión, la interfaz conserva el archivo local sin reportar una nueva lectura. La clave no se imprime ni se guarda en el informe.

Interfaz de comandos. Desde `python/`:

```
..\.venv\Scripts\python.exe main.py list
..\.venv\Scripts\python.exe main.py run --queue all --trace
..\.venv\Scripts\python.exe main.py compare
```

```
..\.venv\Scripts\python.exe main.py astar --input data/processed/cusco_osm_drive.json
```

Listing 8: Comandos principales de la CLI

El siguiente ejemplo recorre generación, validación, ejecución, exportación y verificación. Escribe dos archivos de ejemplo en `python/`, sin alterar los resultados guardados:

```
..\.venv\Scripts\python.exe main.py generate --type random --nodes 40 --seed 3 -o
    ejemplo.json
..\.venv\Scripts\python.exe main.py run -i ejemplo.json -q binary -o resultado.json
..\.venv\Scripts\python.exe main.py verify -i ejemplo.json -r resultado.json
..\.venv\Scripts\python.exe main.py compare -i ejemplo.json
```

Listing 9: Ciclo de prueba con una instancia generada

`graph_io.py` acepta JSON, TXT, CSV o aristas en línea; `run` exporta distancias, rutas y contadores en JSON. Una entrada con peso negativo se rechaza antes de ejecutar Dijkstra. Las mediciones comparativas se obtienen por separado con `reproduce_canonical.py`.

Aplicación web. En dos terminales, desde la raíz y desde `web/`:

```
# Terminal 1, desde la raíz
.\.venv\Scripts\python.exe -m uvicorn web_api:app --app-dir python --host 127.0.0.1 --
    port 8000
# Terminal 2, desde web/
cd web
npm run dev
```

Listing 10: Ejecución de la API FastAPI y de la interfaz React

Abrir http://127.0.0.1:5173/ (o el sitio publicado en GitHub Pages conectado al servicio FastAPI en Render). La interfaz web tiene ocho vistas: Rutas, Instancias, Comparar, Experimentos, Cobertura Cusco, Evidencias, Datos y método y Acerca de. En Rutas se seleccionan red, puntos, costo, algoritmo y cola; se contrastan las tres colas de Dijkstra y A^* por costo y se elige una observación histórica, local o temporal como escenario uniforme. Instancias permite crear o importar grafos; Comparar revisa el banco canónico y agrega consultas de sesión sin mezclarlas; Experimentos mide redes nuevas (tamaños 5 a 200); Cobertura Cusco distingue el snapshot central de 1,5 km del centro ampliado de 3 km, sin presentar ninguno como toda la ciudad; Evidencias y Datos y método reúnen pruebas, bitácora, procedencia, límites y contrato OpenAPI. El plano local no necesita Internet; solo el mapa base necesita teselas. Ninguna consulta opcional sobrescribe las mediciones.

Aplicación móvil Flutter (`mobile/`). Desde `mobile/` se compila la APK de desarrollo con `flutter build apk -debug`. La aplicación apunta por defecto al servicio en Render (https://rutas-cusco-unsaac.onrender.com), admite URL local de emulador (`http://10.0.2.2:8000`) y conserva en su base SQLite interna (`rutas_cusco_local.db`) borradores, instancias exportadas sin recorte, consultas de rutas y experimentos de sesión. Las instrucciones completas se detallan en `docs/MOVIL_FLUTTER.md` y `docs/CONTRATO_MOVIL_API.md`.

Reproducción de tablas y figuras. Desde la raíz, sin alterar las mediciones del 28/10/2026:

```
.\.venv\Scripts\python.exe python/scripts/reproduce_canonical.py --skip-benchmark --
    skip-applied
```

Listing 11: Regeneración de tablas y figuras con mediciones guardadas

El comando anterior reutiliza `benchmark_results.json` y `applied_results.json` y escribe tablas en `latex/tablas/` y figuras en `latex/figuras/`. Sin las opciones de omisión, el script repite las mediciones y *sobrescribe* ambos JSON; ejecutar el protocolo completo solo en una copia separada. La red OSM y el clima ya almacenados no se descargan de nuevo. El piloto del 04/11/2026 se conserva aparte en `pilot_benchmark_results.json`. Para regenerar únicamente su figura desde las muestras guardadas, ejecutar desde la raíz `python python/scripts/reproduce_pilot.py -plot-only`. La imagen se guarda en `latex/figuras/piloto_mediciones.png` y aparece en la Sección 8.

Compilación del informe. Desde `latex/`:

```
.\compilar.ps1
```

Listing 12: Compilación con pdflatex y biber

Anexo C: Cronología verificable del repositorio

La Tabla 22 reúne hitos clave del historial Git en la rama `main` hasta el 12/11/2026. Cada fecha y autor corresponden al registro del commit, no necesariamente al momento de estudio o a la persona que redactó cada línea. No se deducen reuniones, horas de dedicación ni lecturas individuales. La bitácora de aprendizaje por estudiante sigue sin constancia separada en el repositorio.

Tabla 22
Hitos fechados comprobables en la historia de Git (`main`)

Autor en Git	Fecha	Commit	Cambios identificables
Choquenaira, Noe	02/10/2026	`24e526f`	`dijkstra.py`, `indexed_min_heap.py`, `cost_model.py` y primera versión del informe.
Porroa, Yeni	04/10/2026	`442ea56`	`d_ary_heap.py`, capítulo de evaluación del stack y tabla comparativa.
Quispe, Romario	04/10/2026	`221bf21`	`fibonacci_heap.py`, `scalability_benchmark.py` y resultados preliminares.
Yaranga, Aldo	04/10/2026	`ee3109d`	`test_dijkstra.py`, cuaderno Colab y exportación de figuras y tablas.
Yaranga, Aldo	22/10/2026	`4b080db`	Integración de avances 4 y 5: `cli.py`, `graph_io.py`, `instance_generator.py`, registros e instancias inválidas.
Choquenaira, Noe	23/10/2026	`1abbfb5`	`a_star.py` y capítulo de aprendizaje.
Porroa, Yeni	23/10/2026	`fbb14f8`	Capítulo de diseño algorítmico y análisis de estructuras.
Quispe, Romario	23/10/2026	`83ea30a`	Actualización de `benchmark_results.json` previa a la consolidación grupal.
Yaranga, Aldo	23/10/2026	`82e7dd1`	`cli.py` y `test_cli.py`.
Choquenaira, Noe F.	28/10/2026	`80e2593`	*Snapshots* OSM y clima, banco canónico de 1 350 tiempos y caso aplicado.
Choquenaira, Noe F.	04/11/2026	`f574216`	Piloto reproducible de 18 tiempos y síntesis comparativa revisada.
Choquenaira, Noe F.	08/11/2026	`327b4d6`	Sincronización de bitácora oficial y evidencias entre las 4 ramas.
Choquenaira, Noe F.	10/11/2026	`afd523a`, `570f602`	API REST FastAPI, cargador OSM/snapping, visor MapLibre en React y capítulos 1, 6, 7, 11 y 12.
Yaranga, Aldo	10/11/2026	`37c1ce1`, `dc0e7e3`	Certificado $O(\|V\|+\|E\|)$, pruebas A^*/Vitest, `instancias_pruebas/`, Colab y capítulo 8.
Quispe, Romario	10/11/2026	`4fe5483`, `8e7d2cc`	Modelo de impedancia, servicios clima/TomTom, vistas Comparar/Experimentos y capítulos 9–10.
Porroa, Yeni	10–11/11	`1fdf102`, `86067fd`	Sincronización de archivos comunes y fusión de `ramaYeni` hacia `main`.
Choquenaira, Noe F.	12/11/2026	`3280038`	Despliegue unificado con `Dockerfile` y `render.yaml` (Render + GitHub Pages).
Choquenaira, Noe F.	12/11/2026	`adbaa59`, `2984d33`	Contrato API `v1.5.0`, base móvil Flutter (`mobile/`) y actualización de evidencias/bitácora.

Tabla 23
Anexo D: matriz de trazabilidad entre objetivos, componentes y evidencia

Objetivo	Componente	Evidencia
OE1	Modelo de grafo y costo temporal	Sección 4; ecuación (17); instancia ilustrativa y red OSM.
OE2	Tres colas y Dijkstra parametrizado	Sección 5; `src/datastructures/`; 61 pruebas de contrato.
OE3	Validación, certificado y suites	Sección 7; 349 pruebas Python + 44 web + 40 móvil (433 total); Bellman, NetworkX y Bellman-Ford.
OE4	Benchmark, caso OSM, A^* y clima	Sección 9; 1 350 muestras; 891 vértices; 823 vs 887 expansiones; factor 1.0691.
AG_C06.01	Aprendizaje autónomo	Sección 2; Anexo C; pendientes individuales en `docs/CONTROL_ENTREGA.md`.
AG_C06.02	Selección de stack	Sección 3; Tabla 3; adopción temprana por verificar.
AG_C06.03	Experimentación y pensamiento crítico	Sección 8; Sección 10; conclusión condicionada.
Web y API	React + FastAPI + Render/Pages	Sección 6; `web/`, `web_api.py`, `render.yaml`; pruebas de API.
Aplicación móvil	Flutter + SQLite interno (`mobile/`)	Sección 6.3; `mobile/lib/`, `mobile/test/` (40 pruebas), APK debug y capturas en Figs. 3–4.

Anexo E: Distribución de commits y contribuciones por integrante

Para reflejar con precisión el aporte de cada estudiante según su rama de trabajo en el repositorio oficial (`main`, 122 commits al 12/11/2026), la Tabla 24 consolida los commits atribuidos a cada integrante mediante su rama de desarrollo (`ramaFRANKI`, `ramaAldo`, `RamaRomario` y `ramaYeni`), asociando tanto los commits firmados con cuentas personales como los commits de consolidación, pruebas y despliegue canalizados a través de su rama. La Tabla 25 resume los módulos trabajados por cada integrante.

Tabla 24
Distribución verificable de los 122 commits en `main` por autoría Git y rama asociada

Autor(es) registrados en Git	Rama asociada	Commits	Alcance comprobable en esos commits
Choquenaira Quispe, Noe Franklin (`ramaFRANKI`)	`ramaFRANKI`	71	Dijkstra, A^*, cargador OSM, API FastAPI, frontend React/MapLibre, aplicación móvil Flutter, contrato `v1.5.0`, despliegue unificado (Docker/Render) y consolidación de entregas (20 individuales + 51 de integración).
Yaranga Achahui, Aldo (`Aldo Yaranga`, `aldobaz01`)	`ramaAldo`	22	Suite `pytest` (AAA), certificado $O(\|V\|+\|E\|)$, CLI, E/S y generadores, `instancias_pruebas/`, Colab, pruebas Vitest y capítulo 8.
Quispe Rimachi, Romario (`Romario Quispe`)	`RamaRomario`	19	Fibonacci Heap, modelo de impedancia, servicios clima/TomTom, vistas Comparar/Experimentos/Condiciones y capítulos 9–10.
Porroa Sivana, Yeni Ruth (`Yeni Porroa`)	`ramaYeni`	10	Montículo 4-ario (`DAryHeap`), evaluación de stack, diseño de estructuras, sincronización y fusión de rama.
Total en `main` (12/11/2026)	**4 ramas**	**122**	Conteo directo de `git log` sobre `main`; no mide horas de esfuerzo ni líneas redactadas fuera de Git.

Tabla 25
Matriz de componentes por integrante y evidencia en el repositorio

Integrante	Componentes principales	Evidencia
Choquenaira, Noe	Dijkstra, heap binario, A^*, cargador OSM, API FastAPI, interfaz React/MapLibre, aplicación móvil Flutter (`mobile/`) y redacción transversal	`24e526f`, `1abbfb5`, `afd523a`, `570f602`, `adbaa59`; Secciones 6 y 6.3.
Porroa, Yeni	Heap 4-ario (`DAryHeap`), evaluación del stack, análisis de estructuras e integración de rama	`442ea56`, `fbb14f8`, `1fdf102`, `86067fd`; `d_ary_heap.py` y Secciones 3 y 5.
Quispe, Romario	Fibonacci Heap, modelo de impedancia, servicios clima/TomTom, benchmark y vistas de comparación/experimentos	`221bf21`, `83ea30a`, `4fe5483`, `8e7d2cc`, `7826748`; módulos en `services/` y Secciones 8–9.
Yaranga, Aldo	Suite de pruebas AAA, certificado de optimalidad, CLI, E/S, instancias de prueba, Colab y pruebas web	`ee3109d`, `4b080db`, `37c1ce1`, `bd9ba70`, `dc0e7e3`; `python/tests/` y Sección 7.

Anexo F: Estado del seguimiento

Al 13/11/2026 el repositorio integra el núcleo Python, pruebas automatizadas en tres plataformas (349 Python, 44 web y 40 móvil, totalizando 433 pruebas), análisis de complejidad, datos del banco canónico y de la consulta OSM, despliegue unificado en Render y GitHub Pages, aplicación móvil Flutter con SQLite interno, APK de desarrollo compilada y verificada en emulador Android con capturas reales integradas en la Sección 6.3. El piloto de 18 muestras se guardó el 04/11, *después* de las mediciones canónicas del 28/10; no se presenta como una etapa previa que no ocurrió. Las muestras cronometradas de los JSON se expresan en microsegundos; los 387.197 segundos del caso OSM corresponden al costo estimado de viaje de la ruta.

Siguen pendientes de acreditación académica las bitácoras originales individuales de fuentes y aprendizajes por integrante, las actas firmadas, la comprobación de instalación en una máquina limpia, la compilación iOS en macOS/Xcode y la prueba de campo de la APK en teléfonos físicos conectados por ADB. La matriz de pendientes y la plantilla están en `docs/CONTROL_ENTREGA.md` y `docs/plantilla_evidencia_individual.md`.